Modul 9: HMVC, Authorization, dan AJAX pada Laravel React Starter Kit

9.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep HMVC dan menerapkannya pada *Laravel React Starter Kit* menggunakan nwidart/laravel-modules, dengan halaman React di dalam setiap modul. 2. Memahami konsep dasar React (komponen, JSX, *props*, *state*, dan *event*) serta peran Inertia.js sebagai penghubung Laravel dan React.
3. Mampu menerapkan *authentication* bawaan *starter kit* dan *authorization* berbasis *role* (admin dan user) menggunakan *middleware*.
4. Mampu membangun CRUD berbasis Inertia (useForm) di dalam modul HMVC React. 5. Mampu membangun CRUD AJAX dengan fetch() dan *upload* berkas di dalam modul HMVC React.

9.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

1. **Perangkat Keras:** komputer atau laptop dengan sistem operasi Windows/Linux/MacOS, prosesor minimal 1 GHz, RAM 4 GB (disarankan 8 GB), dan ruang penyimpanan kosong minimal 3 GB (folder node_modules membutuhkan ruang tambahan).
2. **Perangkat Lunak:**
   - XAMPP atau Laragon (Apache, MySQL, phpMyAdmin).
   - Composer.
   - PHP 8.2 atau lebih baru.
   - Node.js 20 atau lebih baru beserta npm.
   - Laravel Installer.
   - Laravel React *Starter Kit* (React, TypeScript, Inertia.js, Tailwind CSS, shadcn/ui, Fortify).
   - *Package* nwidart/laravel-modules.
   - Pest untuk pengujian otomatis.
   - Visual Studio Code dan Google Chrome (DevTools).
   - *File* contoh untuk uji *upload* (.jpg, .png, .pdf di bawah 2 MB, dan satu .zip).

9.3 Tugas Pendahuluan

1. Apa yang dimaksud dengan HMVC? Jelaskan perbedaannya dengan MVC biasa, lalu sebutkan keuntungan memecah aplikasi menjadi beberapa modul!
2. Pada HMVC React, bagian mana dari MVC yang ditulis dengan React dan bagian mana yang tetap ditulis dengan PHP? Di *folder* mana halaman React milik sebuah modul disimpan? 3. Apa yang dimaksud dengan komponen dan JSX pada React? Sebutkan minimal dua perbedaan penulisan JSX dengan HTML biasa!
4. Jelaskan perbedaan *props* dan *state* pada React, serta fungsi useState() dan useEffect()! Berikan masing-masing satu contoh sederhana.

5. Apa fungsi Inertia.js dalam menghubungkan Laravel dengan React? Jelaskan perbedaan return view() pada Blade dengan return Inertia::render('NamaModul::halaman') pada HMVC React!

9.4 Materi Praktikum

9.4.1 Konsep HMVC pada Laravel React

Bayangkan sebuah aplikasi seperti gedung kampus. Jika semua kegiatan dilakukan di satu aula besar, aula akan penuh dan berantakan. Akan lebih rapi jika gedung dibagi menjadi beberapa ruangan, misalnya ruang administrasi dan ruang pendaftaran. Setiap ruangan memiliki perlengkapan sendiri dan dapat diperbaiki tanpa mengganggu ruangan lain. Itulah ide dasar HMVC (*Hierarchical Model View Controller*). Aplikasi dipecah menjadi beberapa modul (ruangan), dan setiap modul memiliki bagian MVC miliknya sendiri:

• Model → mengurus data di database, misalnya data pendaftar.

• View → tampilan yang dilihat pengguna. Pada modul ini, View ditulis dengan React. • *Controller* → "pengatur lalu lintas" yang menerima permintaan pengguna, mengambil data dari Model, lalu mengirimkannya ke View.

Pada Modul 8, View ditulis dengan Blade. Pada Modul 9, View diganti dengan React, sedangkan Model dan *Controller* tetap ditulis dengan PHP seperti biasa. Jadi, satu modul berisi semua yang dibutuhkan sebuah fitur, mulai dari database sampai tampilan. Untuk membuat modul di Laravel, kita menggunakan *package* nwidart/laravel-modules.

Isi satu modul pada praktikum ini kira-kira seperti berikut. Struktur ini tidak perlu dihafal; cukup pahami bahwa semua *file* milik fitur Pendaftaran berkumpul di satu *folder*:

```
Modules/Pendaftaran/ ← satu modul = satu fitur
├── app/
│   ├── Http/Controllers/ ← Controller (PHP)
│   └── Models/ ← Model (PHP)
├── database/migrations/ ← rancangan tabel database
├── resources/js/pages/ ← View (halaman React .tsx)
└── routes/web.php ← daftar alamat URL modul
```

Cara kerjanya secara sederhana adalah sebagai berikut:

• Pengguna membuka alamat /pendaftaran di *browser*.

• Laravel mencari alamat tersebut di *file* routes/web.php milik modul Pendaftaran. • *Route* memanggil *Controller* milik modul, lalu *Controller* menyiapkan data yang diperlukan.

• *Controller* memanggil Inertia::render('Pendaftaran::index'), artinya "tampilkan halaman index milik modul Pendaftaran".

• *File* Modules/Pendaftaran/resources/js/pages/index.tsx tampil di *browser* sebagai halaman React.

Agar langkah terakhir dapat berjalan, aplikasi perlu diberi tahu bahwa halaman React juga dapat berada di dalam *folder* modul. Pengaturan ini cukup dilakukan satu kali pada

Tahap A. Keuntungan HMVC adalah setiap fitur tersusun rapi di foldernya sendiri, pekerjaan anggota tim tidak saling bentrok, dan fitur baru cukup ditambahkan sebagai modul baru.

9.4.2 Konsep Dasar React

React adalah *library* JavaScript untuk membuat tampilan web. Cara kerja React mirip menyusun mainan LEGO: tampilan dibangun dari kepingan-kepingan kecil yang disebut komponen, misalnya komponen tombol, komponen tabel, dan komponen *form*. Kepingan yang sama dapat dipakai berulang kali di banyak halaman.

Ada lima istilah dasar React yang akan sering ditemui:

- Komponen → sebuah fungsi yang menghasilkan tampilan. Nama komponen selalu diawali huruf kapital, misalnya function Sapaan(). Komponen dipakai seperti tag HTML, yaitu <Sapaan />.
- JSX → cara menulis tampilan di dalam kode JavaScript yang bentuknya mirip HTML. Perbedaannya, atribut class ditulis className dan nilai dari JavaScript ditulis di antara kurung kurawal, misalnya {nama}. *File* React pada praktikum ini berekstensi .tsx (JSX yang ditulis dengan TypeScript).
- *Props* → data yang dikirim ke komponen dari luar, seperti mengisi formulir sebelum diserahkan, misalnya <Sapaan nama="Budi" />. Data dari *Controller* Laravel juga diterima halaman React sebagai *props*.
- *State* → data yang disimpan oleh komponen dan dapat berubah, misalnya isi kotak pencarian. Setiap kali *state* berubah, React otomatis memperbarui tampilan. *State* dibuat dengan useState().
- *Event* → reaksi terhadap aksi pengguna, misalnya onClick (saat tombol diklik) dan onChange (saat isi input berubah).

Contoh paling sederhana adalah sebuah tombol yang menghitung berapa kali tombol tersebut diklik:

```
import { useState } from 'react';

export default function Penghitung() {
 // state "jumlah" dimulai dari 0
 const [jumlah, setJumlah] = useState(0);

 return (
 <div>
 <p>Tombol sudah diklik {jumlah} kali</p>
 <button onClick={() => setJumlah(jumlah + 1)}>Klik saya</button>  </div>
 );
}
```

- useState(0) → membuat *state* jumlah dengan nilai awal 0, sedangkan setJumlah adalah fungsi untuk mengubah nilainya.
- {jumlah} → menampilkan nilai *state* di dalam teks.
- onClick={() => setJumlah(jumlah + 1)} → setiap kali tombol diklik, jumlah bertambah 1

dan React langsung memperbarui teks di layar tanpa memuat ulang halaman. Selain itu, ada tiga teknik yang akan dipakai di halaman modul:

• Menampilkan daftar → data berbentuk *array* ditampilkan dengan .map(), mirip @foreach pada Blade. Setiap baris diberi key yang unik, misalnya key={item.id}.

• Menampilkan sesuai kondisi → mirip @if pada Blade. Contohnya, {isAdmin && Anda admin } artinya teks hanya muncul jika isAdmin bernilai true.

• useEffect → menjalankan kode secara otomatis ketika halaman pertama kali dibuka, misalnya untuk mengambil data dari *server*.

Contoh penggunaan daftar dan kondisi:

```
const daftarProdi = ['Informatika', 'Sistem Informasi', 'Teknik Elektro'];

export default function DaftarProdi({ isAdmin }: { isAdmin: boolean
}) {  return (
 <div>
 {isAdmin && <p>Anda login sebagai admin</p>}
 <ul>
 {daftarProdi.map((prodi) => (
 <li key={prodi}>{prodi}</li>
 ))}
 </ul>
 </div>
 );
}
```

Agar lebih mudah, berikut padanan penulisan Blade (Modul 8) dan React (Modul 9):

Kebutuhan	Blade (Modul 8)	React (Modul 9)
Menampilkan data	{{ $nama }}	{nama}
Perulangan	@foreach ... @endforeach	data.map((item) => ...)
Kondisi	@if ... @endif	{kondisi && ...}
Kerangka halaman	@extends('layout')	<AppLayout> ... </AppLayout>
Pindah halaman	<a href="...">	<Link href="...">
Pesan error	@error('nama')	errors.nama

9.4.3 Inertia.js dan *React Starter Kit*

Laravel berjalan di *server* (PHP), sedangkan React berjalan di *browser* (JavaScript). Keduanya membutuhkan "penerjemah" agar dapat bekerja sama, dan penerjemah itu adalah Inertia.js. Dengan Inertia, kita tetap menulis *route* dan *controller* Laravel seperti biasa. Bedanya, *controller* tidak lagi memanggil view() Blade, melainkan Inertia::render() untuk menampilkan halaman React:

```
// Modul 8 (Blade)
return view('usermanagement::index', ['users' => $users]);

// Modul 9 (React melalui Inertia)
return Inertia::render('UserManagement::index', ['users' => $users]);
```

Data users yang dikirim *controller* akan diterima halaman React sebagai *props*. Selain itu, ada tiga fitur Inertia yang dipakai pada praktikum ini:

- *Link* → pengganti tag <a>. Perpindahan halaman terasa cepat karena *browser* tidak memuat ulang seluruh halaman.
- useForm() → alat bantu untuk mengurus *form*: menyimpan isi input, mengirimkannya ke *server*, dan menampilkan pesan error validasi. Fitur ini dipakai pada modul UserManagement.

- *Shared props* → data yang otomatis tersedia di semua halaman, misalnya data user yang sedang *login* dan pesan berhasil.

React Starter Kit adalah "paket awal" resmi dari Laravel yang sudah berisi React, Inertia, Tailwind CSS, serta halaman *login* dan *register* yang siap pakai, sehingga kita tidak perlu membuat tampilan dari nol. *Folder* yang paling sering digunakan adalah sebagai berikut:

```
resources/js/
 ├── pages/ ← halaman bawaan (dashboard, login, register)
 ├── components/ui/ ← komponen siap pakai (tombol, input, dialog)
 ├── layouts/ ← kerangka halaman (sidebar dan header)
 └── app.tsx ← file awal yang menjalankan React
```

Komponen siap pakai di *folder* components/ui berasal dari shadcn/ui. Cara memakainya cukup dengan import, misalnya import { Button } from '@/components/ui/button', lalu ditulis <Button>Simpan</Button>. Karena semua modul memakai komponen dan kerangka halaman yang sama, tampilan seluruh modul menjadi seragam.

Starter kit juga sudah menyediakan fitur *register*, *login*, dan *logout* menggunakan Laravel Fortify. Ada dua istilah penting yang perlu dibedakan:

- *Authentication* → memeriksa "siapa kamu?". Contohnya, *login* menggunakan email dan password.
- *Authorization* → memeriksa "kamu boleh melakukan apa?". Contohnya, user biasa boleh mengisi pendaftaran, tetapi hanya admin yang boleh mengelola user.

Untuk *authorization*, kita membuat *middleware* role. *Middleware* ibarat satpam di pintu ruangan: sebelum pengguna masuk, satpam memeriksa kartu (role) miliknya. Jika role tidak sesuai, pengguna ditolak dengan pesan 403 (dilarang masuk). Menyembunyikan menu di React hanya mengatur tampilan, sedangkan penjaga yang sebenarnya tetap *middleware* di *server*.

Hak akses yang diterapkan pada praktikum ini adalah sebagai berikut:

Halaman	Tamu	User	Admin
Login dan Register	Ya	Tidak (dialihkan ke dashboard)	Tidak (dialihkan ke dashboard)
Dashboard	Tidak (ke login)	Ya	Ya
Modul Pendaftaran (CRUD AJAX)	Tidak (ke login)	Ya	Ya
Modul UserManagement (kelola user)	Tidak (ke login)	Tidak (403)	Ya

Pada modul Pendaftaran, data dapat disimpan, diubah, dan dihapus tanpa berpindah atau memuat ulang halaman. Teknik ini disebut AJAX (Asynchronous JavaScript and XML). Analoginya seperti memesan makanan di restoran: kita tetap duduk di meja (halaman tidak berpindah), pelayan membawa pesanan ke dapur (*server*), lalu kembali membawa hasilnya (data). Istilah yang perlu diketahui:

- fetch() → fungsi bawaan *browser* untuk mengirim permintaan ke *server* dari JavaScript. Dalam analogi tadi, fetch() adalah pelayannya.
- JSON → format data yang dikirim balik oleh *server*, contohnya { "nama": "Budi" }. *Controller* mengirimnya dengan response()->json().
- Memperbarui *state* → setelah data diterima, *state* React diubah sehingga tabel di layar langsung berubah.
- Token CSRF → "tanda pengenal" agar *server* yakin bahwa permintaan berasal dari aplikasi kita sendiri. Tanpa token ini, Laravel menolak permintaan dengan error 419. Token dikirim otomatis oleh *file* bantu http.ts yang dibuat pada Tahap E.
- FormData → wadah untuk mengirim isi *form* beserta *file* (berkas *upload*) melalui AJAX.

Detail teknis lainnya, misalnya kode error 422 untuk validasi dan pengiriman _method=PUT, akan dijelaskan langsung saat menulis kode pada Tahap E.

9.4.4 Rancangan Aplikasi

Pada praktikum ini dibuat satu *project* bernama **pertemuan9_hmvc_react** dengan pembagian sebagai berikut:

Bagian	Letak	Jenis	Akses

Register, Login, Logout, Dashboard	Di luar HMVC (Fortify, app/, resources/js/pages)	Halaman React bawaan Starter Kit	Tamu / semua yang login
Middleware role	Di luar HMVC (app/Http/Middlewa re)	Authorization	-
UserManagement	Modules/UserMana gement	CRUD Inertia (form + redirect)	Admin
Pendaftaran	Modules/Pendaftara n	CRUD AJAX (fetch + JSON, 5 item + upload berkas)	Admin dan User

Modul Pendaftaran menyimpan lima item data, yaitu nama, asal sekolah, prodi tujuan, email, dan nomor HP, ditambah satu berkas pendukung (jpg, jpeg, png, atau pdf) yang dapat diunggah, dibuka, diganti, dan dihapus. Modul UserManagement mengelola nama, email, role, dan *password* pengguna.

9.5 Membangun Aplikasi HMVC React

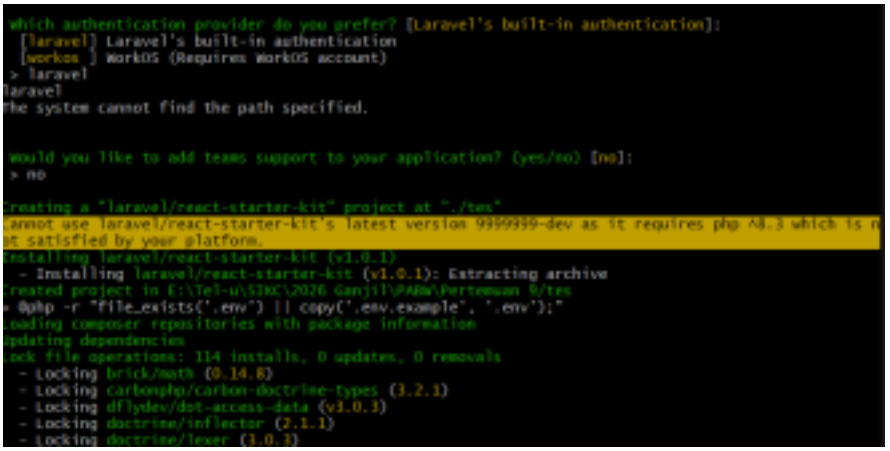
Subbab ini membangun aplikasi dari awal. Tahap A sampai E berisi pembuatan kode, sedangkan seluruh pengujian melalui *browser* dilakukan pada Tahap F. Pengujian digabung di akhir karena *sidebar* aplikasi memanggil halaman dari kedua modul, sehingga aplikasi baru dapat diuji dengan lengkap setelah semua *route* dan halaman React tersedia.

Tahap A: Menyiapkan Project React Starter Kit dan Memasang nwidart/laravel-modules 1. Buka terminal di *folder* tempat menyimpan *project* (misalnya C:\xampp\htdocs), pastikan Apache dan MySQL pada XAMPP/Laragon sudah berjalan, lalu buat *project* Laravel baru menggunakan React Starter Kit:

```
composer global require laravel/installer
laravel new pertemuan9_hmvc_react
```

Jawab pertanyaan Laravel *Installer* sebagai berikut: pilih **React** ketika ditanya *starter kit*, pilih **Laravel's built-in authentication** ketika ditanya penyedia *authentication*, pilih **Pest** ketika ditanya *framework testing*, lalu jawab **Yes** ketika ditanya apakah npm install dan npm run build akan dijalankan.

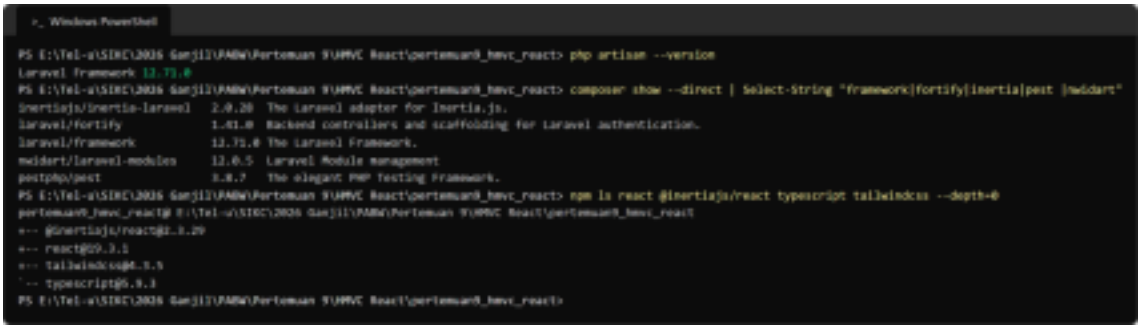




Setelah selesai, masuk ke *folder project*:

```
cd pertemuan9_hmvc_react
code .
```

Jika tidak menggunakan Laravel *Installer*, *project* yang sama dapat dibuat dengan perintah composer create-project laravel/react-starter-kit pertemuan9_hmvc_react, kemudian jalankan npm install di dalam *folder project*.



Gambar 9.1 Pilihan React Starter Kit pada Laravel Installer

2. Buka phpMyAdmin (http://localhost/phpmyadmin), buat *database* baru bernama **hmvc_react**, lalu ubah konfigurasi pada file .env:

```
APP_LOCALE=id

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=hmvc_react
DB_USERNAME=root
DB_PASSWORD=
```

- APP_LOCALE=id → tanggal dan teks seperti "2 menit yang lalu" ditampilkan dalam Bahasa Indonesia.
- DB_CONNECTION=mysql → mengganti *database* bawaan SQLite menjadi MySQL/MariaDB pada XAMPP. Hapus tanda # di depan baris DB_HOST sampai DB_PASSWORD jika masih berupa komentar. *Migration* dijalankan ulang ke MySQL pada Tahap F.

3. Izinkan *merge plugin*, lalu pasang *package* nwidart/laravel-modules:

```
composer config allow-plugins.wikimedia/composer-merge-plugin
true composer require nwidart/laravel-modules
```

- *Package* nwidart/laravel-modules membutuhkan wikimedia/composer-merge-plugin. Perintah composer config allow-plugins... memberi izin agar *plugin* tersebut boleh dijalankan oleh Composer. Jika langkah ini dilewati, Composer akan menanyakan izin secara interaktif (jawab y).

4. Publikasikan file konfigurasi dan *stub* milik *package*:

```
php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider"
```

Perintah ini membuat file config/modules.php, *folder* stubs/nwidart-stubs (*template* file modul), dan file vite-module-loader.js. File vite-module-loader.js tidak digunakan pada modul ini, karena halaman React milik modul dimuat langsung oleh Vite aplikasi utama (diatur pada langkah 9).

5. Buka file composer.json, lalu tambahkan bagian merge-plugin di dalam extra agar *class* di dalam setiap modul ter-*autoload*:

```
"extra": {
"laravel": {
"dont-discover": []
},
"merge-plugin": {
"include": [
```

```
"Modules/*/composer.json"
 ]
 }
 },
```

- Sejak versi 11, *autoload* "Modules\\": "Modules/" pada bagian psr-4 tidak perlu ditambahkan lagi. Setiap modul memiliki composer.json sendiri yang digabungkan melalui merge-plugin.
- Jalankan composer dump-autoload setiap kali selesai mengubah composer.json.



Gambar 9.2 Konfigurasi merge-plugin pada composer.json

6. Buat dua modul sekaligus, yaitu UserManagement dan Pendaftaran, lalu muat ulang *autoload*:

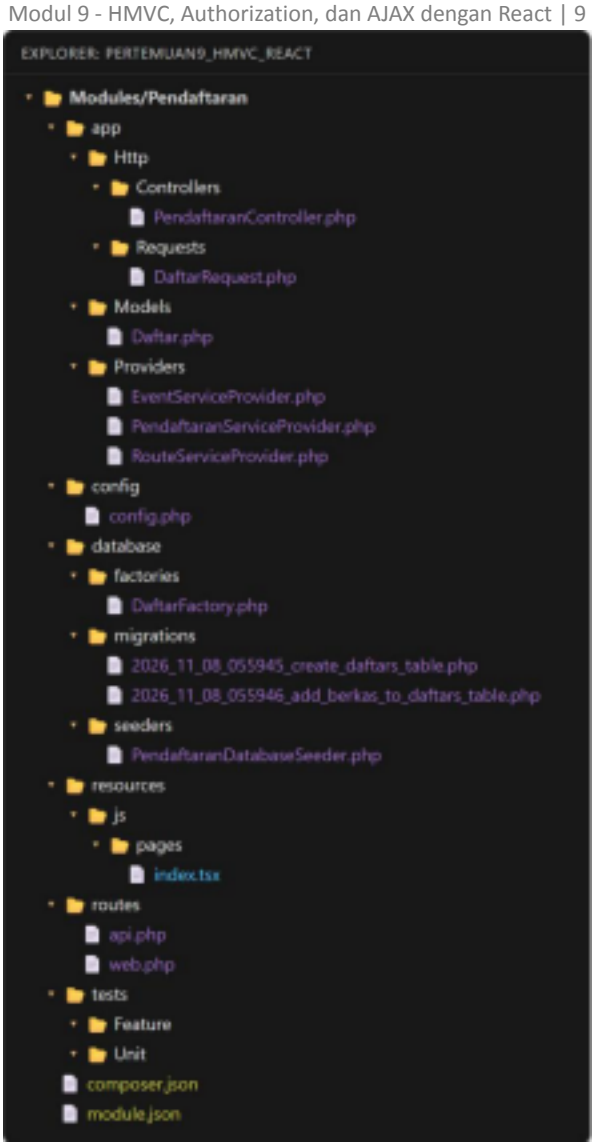
```
php artisan module:make UserManagement Pendaftaran
composer dump-autoload
```

Perintah module:make membuat *folder* Modules/UserManagement dan Modules/Pendaftaran lengkap dengan *service provider*, *controller*, *route*, *view*, serta file module.json dan composer.json milik modul. Nama modul juga dicatat pada file modules_statuses.json dengan status true (aktif).

Karena tampilan modul ditulis dengan React, beberapa file bawaan modul yang berbasis Blade dan Vite milik modul tidak digunakan. Pada kedua modul, hapus *folder* resources/views dan resources/assets, serta file vite.config.js dan package.json. Setelah itu buat *folder* tempat halaman React milik modul:

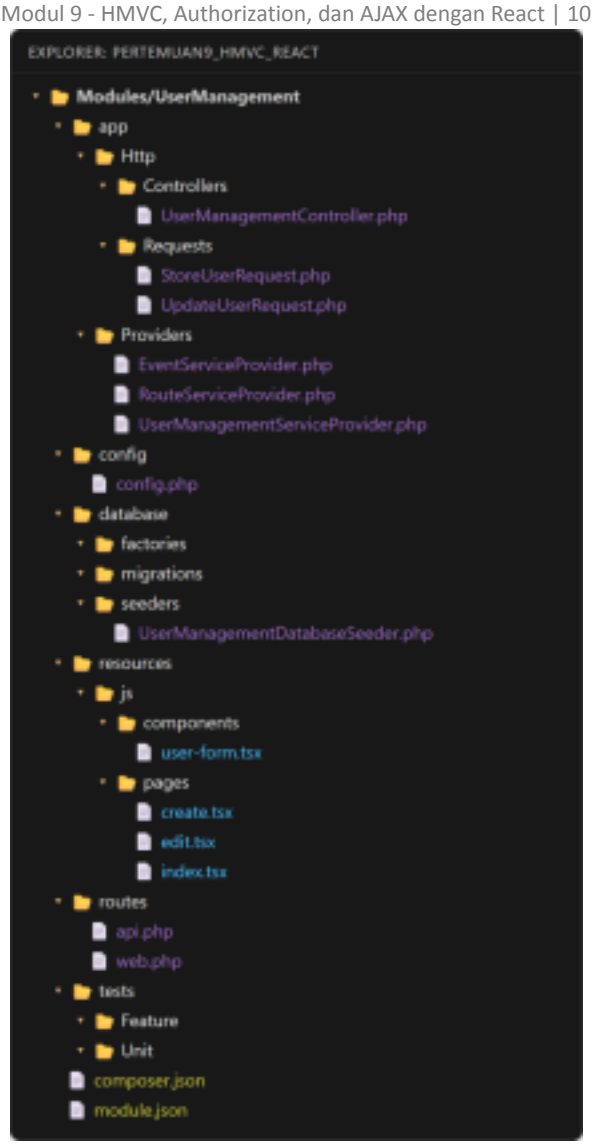
```
mkdir Modules\UserManagement\resources\js\pages
mkdir Modules\Pendaftaran\resources\js\pages
```

- Pada Linux/MacOS gunakan perintah mkdir -p Modules/UserManagement/resources/js/pages (dan seterusnya), atau buat *folder* melalui *Explorer* Visual Studio Code.



Gambar 9.3 Struktur folder Modules/Pendaftaran

Struktur akhir modul Pendaftaran setelah seluruh tahap selesai. File *controller*, *request*, *model*, *factory*, *migration*, *route*, dan halaman React berada di dalam *folder* modul.



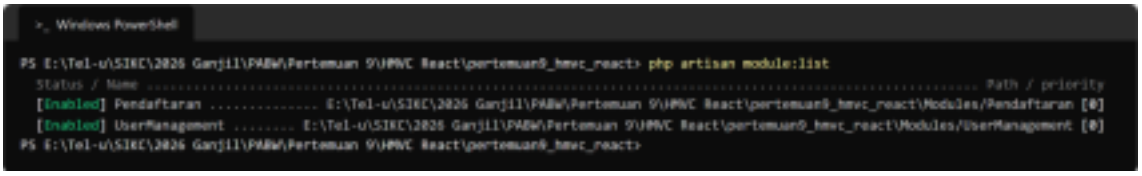
Gambar 9.4 Struktur folder Modules/UserManagement

Modul UserManagement tidak memiliki *model* sendiri karena menggunakan *model*

App\Models\User milik aplikasi utama.

7. Periksa modul yang sudah terdaftar dengan perintah berikut:

```
php artisan module:list
```



Gambar 9.5 Hasil php artisan module:list

Kedua modul berstatus *Enabled* sehingga *route*-nya dimuat oleh Laravel. 8. Kosongkan file routes/api.php pada kedua modul, yaitu Modules/UserManagement/routes/api.php dan Modules/Pendaftaran/routes/api.php:

```
<?php

// Modul ini tidak menyediakan endpoint API.
```


- *Route* API bawaan modul menggunakan *middleware* auth:sanctum, padahal Laravel Sanctum tidak dipasang pada *project* ini. Selain itu *route* tersebut menunjuk method *controller* yang tidak digunakan. Karena modul ini tidak menyediakan API, file cukup dikosongkan.

9. Atur Inertia agar dapat menemukan halaman React di dalam modul. Buka file resources/js/app.tsx, lalu ganti bagian resolve di dalam createInertiaApp() dengan kode berikut (bagian lain bawaan *starter kit* dibiarkan seperti semula):

```
import { createInertiaApp } from '@inertiajs/react';
import { resolvePageComponent } from
'laravel-vite-plugin/inertia-helpers'; // ... import lain bawaan starter
kit dibiarkan seperti semula

createInertiaApp({
 // ... title bawaan starter kit
 resolve: (name) => {
 const pages = import.meta.glob([
 './pages/**/*.tsx',
 '../../Modules/*/resources/js/pages/**/*.tsx',
 ]);

 // Halaman modul ditulis dengan format NamaModul::nama-halaman  if
(name.includes('::')) {
 const [module, page] = name.split('::');

 return resolvePageComponent(
 `../../Modules/${module}/resources/js/pages/${page}.tsx`,  pages,
 );
 }

 return resolvePageComponent(`./pages/${name}.tsx`, pages);  },
 // ... setup dan progress bawaan starter kit
});
```

- import.meta.glob([...]) → Vite mengumpulkan seluruh halaman React, baik milik aplikasi utama (resources/js/pages) maupun milik setiap modul (Modules/*/resources/js/pages).
- name.includes('::') → jika *controller* memanggil Inertia::render('Pendaftaran::index'), nama tersebut dipecah menjadi Pendaftaran dan index, lalu file Modules/Pendaftaran/resources/js/pages/index.tsx dimuat. Tanpa tanda ::, halaman dicari di resources/js/pages seperti biasa, misalnya dashboard dan auth/login.
- Jika file resources/js/ssr.tsx juga digunakan (*server-side rendering*), lakukan perubahan resolve yang sama pada file tersebut.



Gambar 9.6 Pengaturan resolve halaman modul pada app.tsx

10. Daftarkan *folder* modul pada Tailwind CSS agar *class* Tailwind yang ditulis di halaman modul ikut dibuat. Buka file resources/css/app.css, lalu tambahkan baris berikut di bawah baris @source yang sudah ada:

```
@source '../../Modules/*/resources/js/**/*.{ts,tsx}';
```

- Tailwind CSS 4 hanya membuat CSS untuk *class* yang ditemukan pada file yang dipindai. Tanpa baris ini, halaman modul tampil tanpa *style* (tabel dan tombol terlihat berantakan).

11. Agar Visual Studio Code mengenali TypeScript di dalam modul, buka file tsconfig.json lalu tambahkan *folder* modul pada bagian include:

```
"include": [
"resources/js/**/*.ts",
"resources/js/**/*.d.ts",
"resources/js/**/*.tsx",
"Modules/*/resources/js/**/*.ts",
"Modules/*/resources/js/**/*.tsx"
]
```

- Alias @/ (misalnya @/components/ui/button) menunjuk ke *folder* resources/js milik aplikasi utama, sehingga halaman modul dapat memakai komponen, *layout*, dan *hook* bawaan *starter kit*.

Tahap B: Role Pengguna dan Authentication Bawaan Starter Kit di Luar HMVC

Berbeda dengan Modul 8 yang membuat *register*, *login*, dan *logout* secara manual, React Starter Kit sudah menyediakan fitur tersebut melalui Laravel Fortify beserta halaman React-nya. Fitur ini tetap berada di folder app/ dan resources/js/pages (di luar modul) karena digunakan oleh seluruh modul. Pada tahap ini kita menambahkan kolom role, menyesuaikan proses *register*, serta menyiapkan data dan komponen bersama yang dipakai oleh halaman modul. 1. Buat *migration* untuk menambahkan kolom role pada tabel users:

```
php artisan make:migration add_role_to_users_table --table=users
```

Buka file database/migrations/xxxx_xx_xx_xxxxxx_add_role_to_users_table.php, lalu ganti isinya dengan kode berikut:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
 /**
 * Run the migrations.
 */
 public function up(): void
 {
 Schema::table('users', function (Blueprint $table) {
 $table->string('role')->default('user')->after('email');  });
 }

 /**
 * Reverse the migrations.
 */
 public function down(): void
 {
 Schema::table('users', function (Blueprint $table) {
 $table->dropColumn('role');
 });
 }
};
```

- string('role')->default('user') → setiap akun baru otomatis berperan sebagai user jika role tidak diisi.
- after('email') → kolom role diletakkan setelah kolom email (berlaku pada MySQL).

- dropColumn('role') pada method down() → menghapus kolom ketika *migration* di *rollback*.



Gambar 9.7 Migration penambahan kolom role

2. Buat enum untuk daftar role:

```
php artisan make:enum Enums/UserRole --string
```

Buka file app/Enums/UserRole.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Enums;

enum UserRole: string
{
 case Admin = 'admin';
 case User = 'user';

 /**
 * Get the human readable label of the role.
 */
 public function label(): string
 {
 return match ($this) {
 self::Admin => 'Admin',
 self::User => 'User',
 };
 }
}
```


• enum UserRole: string → enum dengan nilai teks (admin dan user) yang tersimpan di

kolom role.

- label() → mengembalikan nama role yang ramah dibaca. Nilai ini dikirim ke React untuk ditampilkan pada *badge* role dan pilihan role pada form.



Gambar 9.8 Enum UserRole

3. Buka file app/Models/User.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Fortify\TwoFactorAuthenticatable;

class User extends Authenticatable
{
 /** @use HasFactory<UserFactory> */
 use HasFactory, Notifiable, TwoFactorAuthenticatable;

 /**
 * The attributes that are mass assignable.
 *
 * @var list<string>
 */
 protected $fillable = [
 'name',
 'email',
 'role',
 'password',
 ];

 /**
 * The attributes that should be hidden for serialization.
```

```
 *
 * @var list<string>
 */
 protected $hidden = [
 'password',
 'two_factor_secret',
 'two_factor_recovery_codes',
 'remember_token',
 ];

 /**
 * The model's default attribute values.
 *
 * @var array<string, mixed>
 */
 protected $attributes = [
 'role' => 'user',
 ];

 /**
 * Get the attributes that should be cast.
 *
 * @return array<string, string>
 */
 protected function casts(): array
 {
 return [
 'email_verified_at' => 'datetime',
 'password' => 'hashed',
 'two_factor_confirmed_at' => 'datetime',
 'role' => UserRole::class,
 ];
 }

 /**
 * Determine whether the user has the admin role.
 */
 public function isAdmin(): bool
 {
 return $this->role === UserRole::Admin;
 }

 /**
 * Determine whether the user has one of the given roles.
 */
 public function hasRole(UserRole|string ...$roles): bool
 {
 foreach ($roles as $role) {
 if ($this->role === ($role instanceof UserRole ? $role :
UserRole::tryFrom($role))) {
 return true;
 }
 }

 return false;
 }
}
```

• 'role' pada $fillable → kolom role boleh diisi melalui User::create() dan update(). • $attributes = ['role' => 'user'] → nilai bawaan role pada objek User sebelum disimpan. • 'role' => UserRole::class pada casts() → nilai role otomatis diubah menjadi enum UserRole, sehingga dapat dibandingkan dengan UserRole::Admin. Ketika dikirim

ke React, enum otomatis berubah kembali menjadi teks admin atau user.

• TwoFactorAuthenticatable serta kolom two_factor_secret, two_factor_recovery_codes, dan two_factor_confirmed_at adalah bawaan React Starter Kit untuk fitur *two-factor authentication*, sehingga tidak boleh dihapus.

• isAdmin() → memeriksa apakah pengguna adalah admin. Method ini dipakai oleh *controller* dan Form Request.

• hasRole(...$roles) → memeriksa apakah role pengguna termasuk salah satu role yang diberikan. Method ini dipakai oleh *middleware* role.



Gambar 9.9 Model User dengan role

4. Buka file database/factories/UserFactory.php, lalu tambahkan kolom role pada method definition() serta method admin() agar *factory* dapat membuat akun admin. Baris lain bawaan *starter kit* dibiarkan seperti semula:

```
use App\Enums\UserRole;
```

```
public function definition(): array
 {
 return [
 'name' => fake()->name(),
 'email' => fake()->unique()->safeEmail(),
 'role' => UserRole::User,
 'email_verified_at' => now(),
 'password' => static::$password ??= Hash::make('password'),
'remember_token' => Str::random(10),
 // ... kolom bawaan starter kit lainnya (jika ada) dibiarkan  ];
 }

 /**
 * Indicate that the user is an admin.
 */
 public function admin(): static
 {
 return $this->state(fn (array $attributes) => [
 'role' => UserRole::Admin,
 ]);
 }
```

• 'role' => UserRole::User → akun buatan *factory* secara bawaan berperan sebagai user. • admin() → *state factory* untuk membuat akun admin, misalnya User::factory()->admin()->create().



Gambar 9.10 State admin pada UserFactory

5. Buka file database/seeders/DatabaseSeeder.php, lalu ganti isinya dengan kode berikut untuk membuat akun awal:

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
 use WithoutModelEvents;

 /**
 * Seed the application's database.
 */
 public function run(): void
 {
 User::factory()->admin()->create([
```

```
'name' => 'Administrator',
'email' => 'admin@example.com',
'password' => 'password',
]);

User::factory()->create([
'name' => 'User Biasa',
'email' => 'user@example.com',
'password' => 'password',
]);
}
}
```

- *Seeder* membuat satu akun admin (**admin@example.com**) dan satu akun user (**user@example.com**), keduanya dengan *password* **password**.
- *Password* ditulis biasa, lalu otomatis di-*hash* oleh cast 'password' => 'hashed' pada *model* User sebelum disimpan.

DatabaseSeeder.php

database › seeders › DatabaseSeeder.php

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

Gambar 9.11 DatabaseSeeder dengan akun admin dan user

6. Sesuaikan proses *register* bawaan Fortify agar setiap akun baru berperan sebagai user. Buka file app/Actions/Fortify/CreateNewUser.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace App\Actions\Fortify;

use App\Enums\UserRole;
use App\Models\User;
```



```php
use Illuminate\Support\Facades\Validator;
use Illuminate\Validation\Rules\Password;
use Laravel\Fortify\Contracts\CreatesNewUsers;

class CreateNewUser implements CreatesNewUsers
{
 /**
 * Validate and create a newly registered user.
 * Self-registered accounts always get the "user" role.
 *
 * @param array<string, string> $input
 */
 public function create(array $input): User
 {
 Validator::make($input, [
 'name' => ['required', 'string', 'max:255'],
 'email' => ['required', 'string', 'email', 'max:255',
'unique:users,email'],
 'password' => ['required', 'string', 'confirmed', Password::defaults()],
])->validate();

 return User::create([
 'name' => $input['name'],
 'email' => $input['email'],
 'password' => $input['password'],
 'role' => UserRole::User,
 ]);
 }
}
```

- CreateNewUser dipanggil oleh Fortify setiap kali form *register* pada halaman React resources/js/pages/auth/register.tsx dikirim. Class ini menggantikan RegisterRequest dan RegisterController pada Modul 8.

• unique:users,email → email yang sama tidak dapat didaftarkan dua kali. • confirmed → *input* password harus sama dengan *input* password_confirmation yang sudah tersedia pada halaman *register*.

• Password::defaults() → aturan *password* bawaan Laravel (minimal 8 karakter). • **'role' => UserRole::User** → akun hasil *register* selalu berperan sebagai user. Field role tidak dibaca dari *input*, sehingga pengguna tidak dapat mendaftarkan dirinya sebagai admin. Akun admin hanya dapat dibuat melalui *seeder* atau oleh admin di modul UserManagement.

- Setelah akun dibuat, Fortify otomatis melakukan *login* dan mengarahkan pengguna ke /dashboard (diatur oleh nilai home pada config/fortify.php).

CreateNewUser.php

app › Actions › Fortify › CreateNewUser.php

```php
<?php

namespace App\Actions\Fortify;

use App\Enums\UserRole;
use App\Models\User;
use Illuminate\Support\Facades\Validator;
use Illuminate\Validation\Rules\Password;
use Laravel\Fortify\Contracts\CreatesNewUsers;

class CreateNewUser implements CreatesNewUsers
{
    /**
     * Validate and create a newly registered user.
     * Self-registered accounts always get the "user" role.
     *
     * @param  array<string, string>  $input
     */
    public function create(array $input): User
    {
        Validator::make($input, [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'password' => ['required', 'string', 'confirmed', Password::defaults()],
        ])->validate();

        return User::create([
            'name' => $input['name'],
            'email' => $input['email'],
            'password' => $input['password'],
            'role' => UserRole::User,
        ]);
    }
}
```

Gambar 9.12 CreateNewUser dengan role user

7. Pahami proses *login* dan *logout* bawaan Fortify. Bagian ini tidak perlu diubah, tetapi perlu dipahami karena menggantikan LoginController pada Modul 8:
    - app/Providers/FortifyServiceProvider.php → menghubungkan Fortify dengan halaman React, misalnya Fortify::loginView() merender halaman auth/login dan Fortify::registerView() merender halaman auth/register melalui Inertia::render().
    - RateLimiter::for('login', ...) pada FortifyServiceProvider → membatasi percobaan

login maksimal 5 kali per menit untuk kombinasi email dan alamat IP guna mencegah tebak tebakan *password* (*brute force*).

• Fortify memeriksa email dan *password* ke tabel users, membuat ID *session* baru setelah *login* berhasil (mencegah *session fixation*), lalu mengembalikan pengguna ke halaman yang tadi ingin dibuka atau ke /dashboard.

• Logout dikirim dengan method POST ke /logout dari menu pengguna di bagian bawah *sidebar*. Fortify menghapus seluruh data *session* dan membuat token CSRF baru.

8. Pesan *error login* bawaan Fortify masih berbahasa Inggris. Buat file lang/id/auth.php agar pesan tersebut tampil dalam Bahasa Indonesia sesuai APP_LOCALE=id:

```
<?php
```



```
return [
 'failed' => 'Email atau password salah.',
 'password' => 'Password yang dimasukkan salah.',
 'throttle' => 'Terlalu banyak percobaan login. Coba lagi dalam :seconds detik.',
];
```

• failed → pesan ketika email atau *password* tidak cocok, sedangkan throttle → pesan ketika percobaan *login* melebihi batas. Tanda :seconds otomatis diganti dengan sisa waktu tunggu.

• Jika *folder* lang belum ada, buat *folder* lang/id secara manual. Pesan validasi lain yang tidak diterjemahkan tetap tampil dalam Bahasa Inggris.

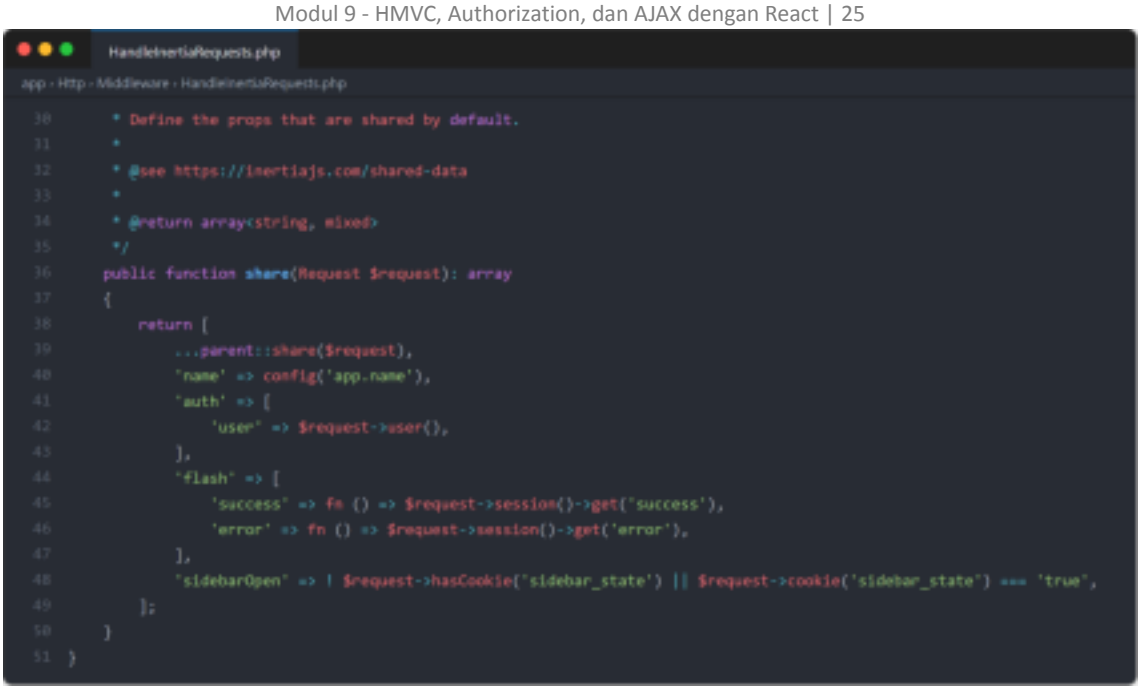
9. Bagikan pesan *flash* dari *session* ke seluruh halaman React. Buka file app/Http/Middleware/HandleInertiaRequests.php, lalu tambahkan *key* flash pada method share(). *Key* lain bawaan *starter kit* dibiarkan seperti semula:

```
public function share(Request $request): array
 {
 return [
 ...parent::share($request),
 'name' => config('app.name'),
 'auth' => [
 'user' => $request->user(),
 ],
 'flash' => [
 'success' => fn () => $request->session()->get('success'),  'error' =>
fn () => $request->session()->get('error'),  ],
 // ... key lain bawaan starter kit (misalnya sidebarOpen) dibiarkan  ];
 }
```

• auth.user → data pengguna yang sedang *login*, termasuk kolom role, dikirim ke setiap halaman. Inilah pengganti auth()->user() pada Blade.

• flash.success dan flash.error → pesan dari redirect()->with('success', ...) pada *controller* dibaca dari *session*, lalu dikirim sebagai *props* sehingga dapat

ditampilkan oleh React. Pada Modul 8, pesan ini ditampilkan oleh *layout* Blade melalui session('success').



Gambar 9.13 Shared props auth dan flash pada HandleInertiaRequests

10. Buat *hook* untuk membaca *shared props* dengan mudah. Buat file resources/js/hooks/use-shared-props.ts:

```
import { usePage } from '@inertiajs/react';

export type UserRole = 'admin' | 'user';

export type AuthUser = {
 id: number;
 name: string;
 email: string;
 role: UserRole;
};

type SharedProps = {
 auth: { user: AuthUser };
 flash: { success?: string | null; error?: string | null };
};

export function useSharedProps() {
 const { auth, flash } = usePage<SharedProps>().props;

 return {
 user: auth.user,
 isAdmin: auth.user?.role === 'admin',
 flash,
 };
}
```

- usePage().props → membaca seluruh *props* halaman, termasuk *shared props* dari HandleInertiaRequests.
- isAdmin → bernilai true jika role pengguna adalah admin. Nilai ini dipakai untuk menampilkan menu Kelola User. Ini hanya menyembunyikan menu; perlindungan sebenarnya tetap dilakukan oleh *middleware* role di sisi *server*.

11. Buat komponen pesan berhasil dan pesan *error* yang dipakai bersama oleh seluruh modul. Buat file resources/js/components/alert-message.tsx:


```
import { CircleAlert, CircleCheck, X } from 'lucide-react';
import { useEffect, useState } from 'react';
import { useSharedProps } from '@/hooks/use-shared-props';

type AlertMessageProps = {
 type: 'success' | 'error';
 message?: string | null;
 onClose?: () => void;
};

export function AlertMessage({ type, message, onClose }: AlertMessageProps)
{  // Pesan berhasil otomatis tertutup setelah 4 detik
 useEffect(() => {
 if (!message || type !== 'success' || !onClose) {
 return;
 }

 const timer = setTimeout(onClose, 4000);

 return () => clearTimeout(timer);
 }, [message, type]);

 if (!message) {
 return null;
```

```
 }

 const Icon = type === 'success' ? CircleCheck : CircleAlert;
const color =
 type === 'success'
 ? 'border-green-200 bg-green-50 text-green-800 dark:border-green-900
dark:bg-green-950 dark:text-green-200'
 : 'border-red-200 bg-red-50 text-red-800 dark:border-red-900 dark:bg-red 950
dark:text-red-200';

 return (
 <div role="alert" className={`flex items-start gap-3 rounded-lg border px-4  py-3
text-sm ${color}`}>
 <Icon className="mt-0.5 size-4 shrink-0" />
 <p className="flex-1">{message}</p>
 {onClose && (
 <button type="button" onClick={onClose} className="opacity-70
hover:opacity-100" aria-label="Tutup">
 <X className="size-4" />
 </button>
 )}
 </div>
 );
}

export function FlashMessage() {
 const { flash } = useSharedProps();
 const [success, setSuccess] = useState(flash.success);
 const [error, setError] = useState(flash.error);

 // Tampilkan ulang pesan setiap kali server mengirim flash baru
useEffect(() => {
 setSuccess(flash.success);
 setError(flash.error);
 }, [flash]);

 return (
 <>
 <AlertMessage type="success" message={success} onClose={() =>
setSuccess(null)} />
```



```
 <AlertMessage type="error" message={error} onClose={() => setError(null)}  />
 </>
 );
 }
```

- AlertMessage → komponen pesan berwarna hijau (berhasil) atau merah (*error*). Komponen ini dipakai langsung oleh modul Pendaftaran untuk menampilkan pesan dari respons JSON.
- FlashMessage → membaca flash dari *shared props*, lalu menampilkannya dengan AlertMessage. Komponen ini dipakai oleh modul UserManagement yang pesannya berasal dari *redirect*.
- setTimeout(onClose, 4000) → pesan berhasil otomatis tertutup setelah 4 detik, sama seperti *class* .alert-autohide pada Modul 8.

12. Buat komponen dialog konfirmasi hapus yang dipakai bersama oleh kedua modul.

Buat file resources/js/components/confirm-dialog.tsx:

```
import { Button } from '@/components/ui/button';
import {
 Dialog,
 DialogClose,
 DialogContent,
 DialogDescription,
 DialogFooter,
 DialogHeader,
 DialogTitle,
} from '@/components/ui/dialog';

type ConfirmDialogProps = {
 open: boolean;
 title?: string;
 description: string;
 confirmLabel?: string;
 processing?: boolean;
 onConfirm: () => void;
 onOpenChange: (open: boolean) => void;
};

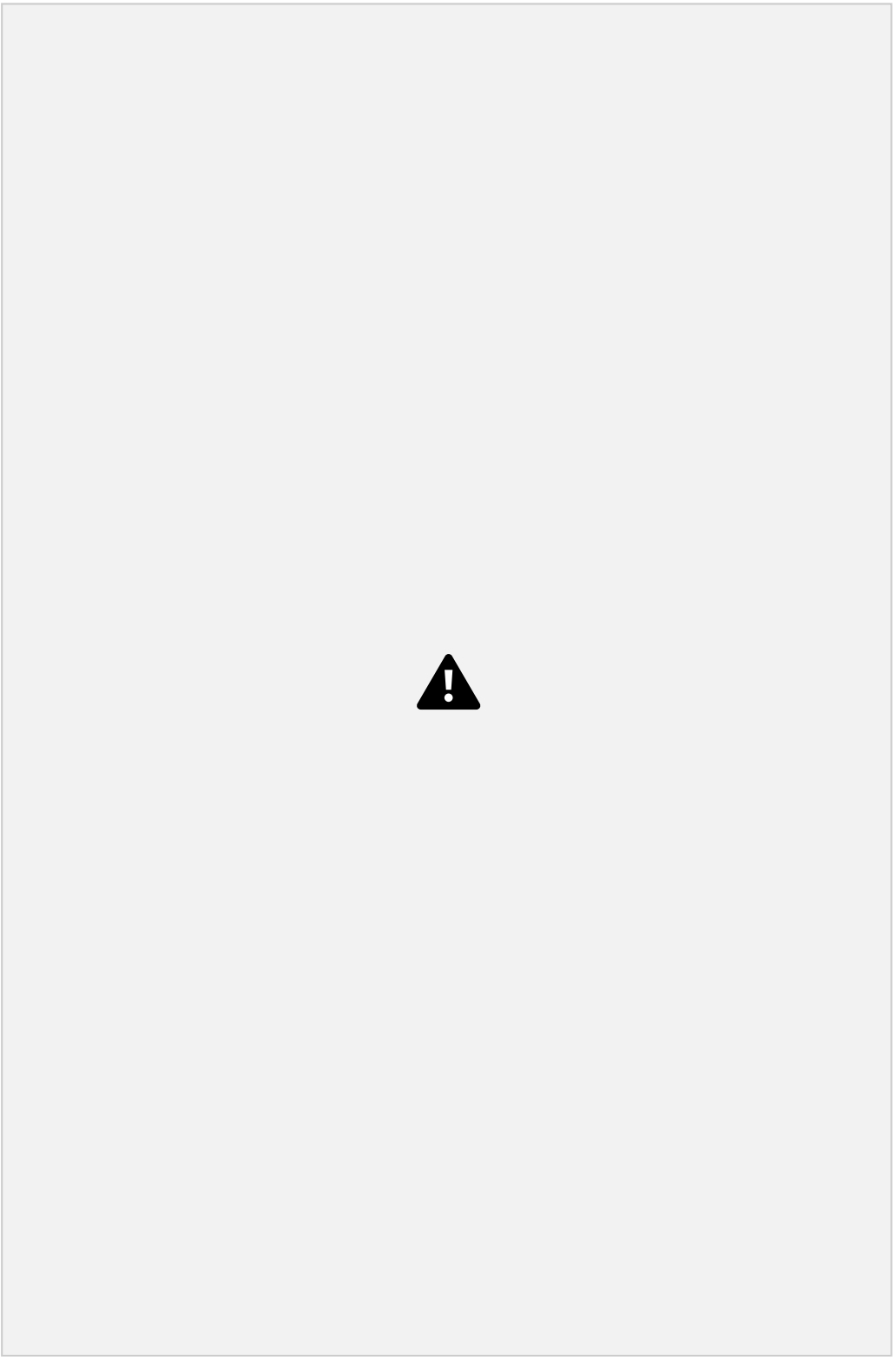
export function ConfirmDialog({
 open,
 title = 'Konfirmasi Hapus',
 description,
 confirmLabel = 'Ya, hapus',
 processing = false,
 onConfirm,
 onOpenChange,
}: ConfirmDialogProps) {
 return (
 <Dialog open={open} onOpenChange={onOpenChange}>
 <DialogContent>
 <DialogHeader>
 <DialogTitle>{title}</DialogTitle>
                  <DialogDescription>{description}</DialogDescription>
 </DialogHeader>
 <DialogFooter className="gap-2">
 <DialogClose asChild>
 <Button variant="secondary" disabled={processing}>  Batal
 </Button>
 </DialogClose>
```



```
 <Button variant="destructive" onClick={onConfirm}
disabled={processing}>
 {processing ? 'Menghapus...' : confirmLabel}  </Button>
 </DialogFooter>
 </DialogContent>
 </Dialog>
 );
}
```

- Dialog, DialogContent, dan komponen lainnya → komponen shadcn/ui bawaan *starter kit* di *folder* resources/js/components/ui. Komponen ini menggantikan modal Bootstrap dan fungsi confirmDelete() pada Modul 8.
  - open dan onOpenChange → dialog dikendalikan oleh *state* milik halaman yang

memakainya, sehingga halaman dapat menentukan data mana yang akan dihapus.



Gambar 9.14 Komponen ConfirmDialog

13. Tambahkan menu modul pada *sidebar*. Buka file resources/js/components/app sidebar.tsx, lalu ganti isinya dengan kode berikut:

```
import { Link } from '@inertiajs/react';
import { ClipboardList, LayoutGrid, Users } from 'lucide-react';
import AppLogo from '@/components/app-logo';
import { NavMain } from '@/components/nav-main';
import { NavUser } from '@/components/nav-user';
```

```
import {
 Sidebar,
 SidebarContent,
 SidebarFooter,
 SidebarHeader,
 SidebarMenu,
 SidebarMenuButton,
 SidebarMenuItem,
} from '@/components/ui/sidebar';
import { useSharedProps } from '@/hooks/use-shared-props';
import type { NavItem } from '@/types';

export function AppSidebar() {
 const { isAdmin } = useSharedProps();

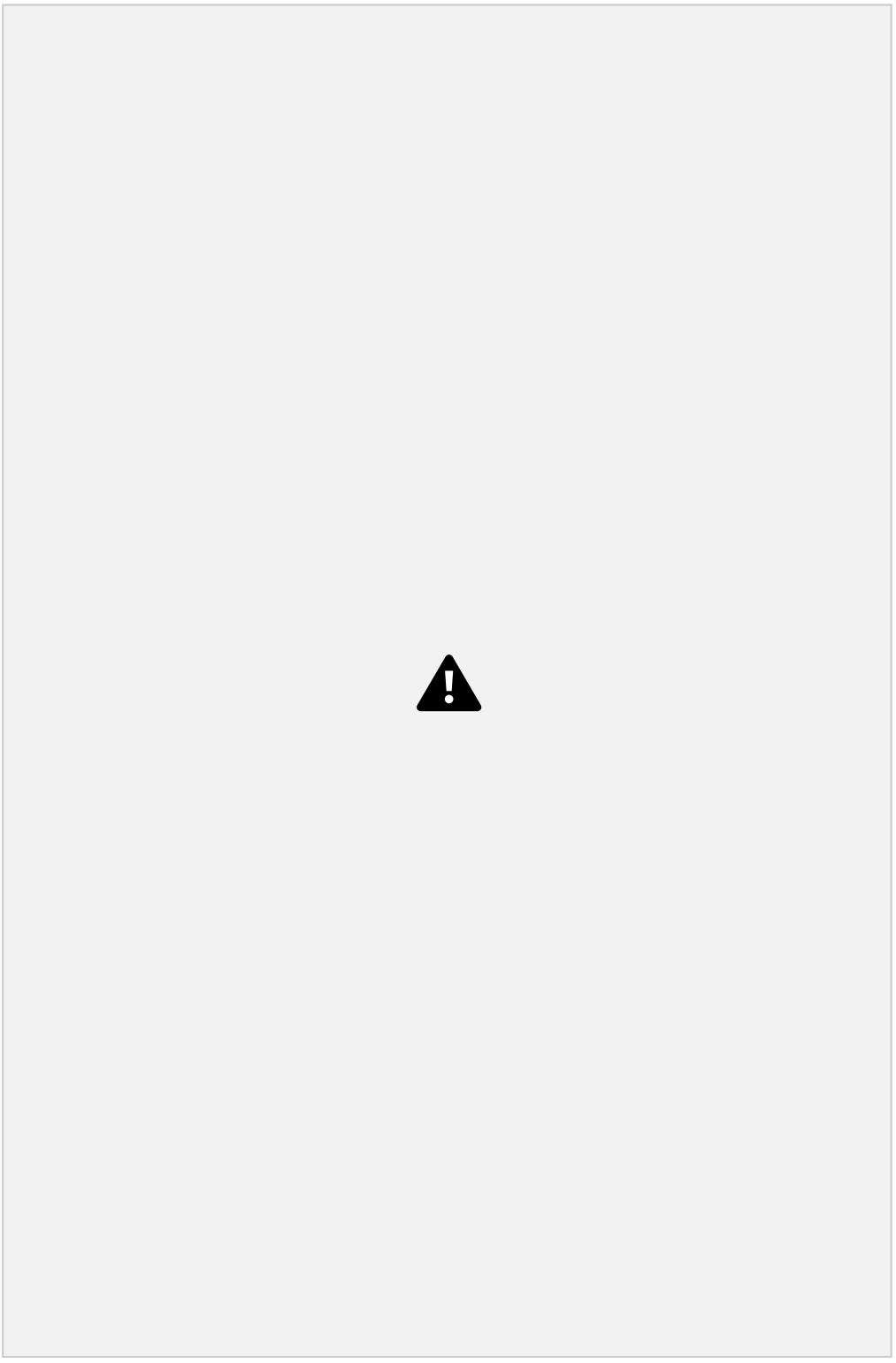
 const mainNavItems: NavItem[] = [
 { title: 'Dashboard', href: '/dashboard', icon: LayoutGrid },  { title:
'Pendaftaran', href: '/pendaftaran', icon: ClipboardList },  ...(isAdmin ? [{
title: 'Kelola User', href: '/admin/users', icon: Users }] :  []),
 ];

 return (
 <Sidebar collapsible="icon" variant="inset">
 <SidebarHeader>
 <SidebarMenu>
 <SidebarMenuItem>
 <SidebarMenuButton size="lg" asChild>
 <Link href="/dashboard" prefetch>
 <AppLogo />
 </Link>
 </SidebarMenuButton>
 </SidebarMenuItem>
 </SidebarMenu>
 </SidebarHeader>

 <SidebarContent>
 <NavMain items={mainNavItems} />
 </SidebarContent>

 <SidebarFooter>
 <NavUser />
 </SidebarFooter>
 </Sidebar>
 );
}
```

- mainNavItems → daftar menu *sidebar*. Menu Kelola User hanya ditambahkan jika isAdmin bernilai true. Bagian ini menggantikan navbar berbasis role pada layouts/app.blade.php di Modul 8.
- NavMain → komponen bawaan yang otomatis memberi tanda aktif pada menu yang sedang dibuka, misalnya menu Pendaftaran ketika URL diawali /pendaftaran (pengganti request()->routeIs()).
- NavUser → menu pengguna di bagian bawah *sidebar* yang berisi nama, email, Settings, dan Log out.
- Tautan Repository dan Documentation bawaan *starter kit* sengaja dihapus agar *sidebar* hanya berisi menu aplikasi.



Gambar 9.15 Menu sidebar berbasis role pada app-sidebar.tsx

14. Buat *controller* untuk halaman *dashboard*:

```
php artisan make:controller DashboardController --invokable
```

Buka file app/Http/Controllers/DashboardController.php, lalu ganti isinya dengan kode berikut:

```
<?php
```



```
namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
 /**
 * Show the dashboard with record counts per module.
 */
 public function __invoke(Request $request): Response
 {
 return Inertia::render('dashboard', [
 'totalPendaftar' => Daftar::count(),
 'totalUsers' => $request->user()->isAdmin() ? User::count() : null,  ]);
 }
}
```

- *Controller invokable* hanya memiliki satu method __invoke(), cocok untuk halaman yang hanya menampilkan satu tampilan.
- Inertia::render('dashboard', [...]) → merender halaman resources/js/pages/dashboard.tsx dan mengirim data totalPendaftar serta totalUsers sebagai *props*. Jumlah user hanya dihitung untuk admin, sedangkan untuk user nilainya null.
- Dashboard menampilkan jumlah pendaftar dari *model* Daftar milik modul Pendaftaran (dibuat pada Tahap E).

15. Buka file routes/web.php, lalu ganti *route dashboard* bawaan agar memakai DashboardController:

```
<?php

use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;
use Laravel\Fortify\Features;

Route::get('/', function () {
 return Inertia::render('welcome', [
 'canRegister' => Features::enabled(Features::registration()),
]);
})->name('home');

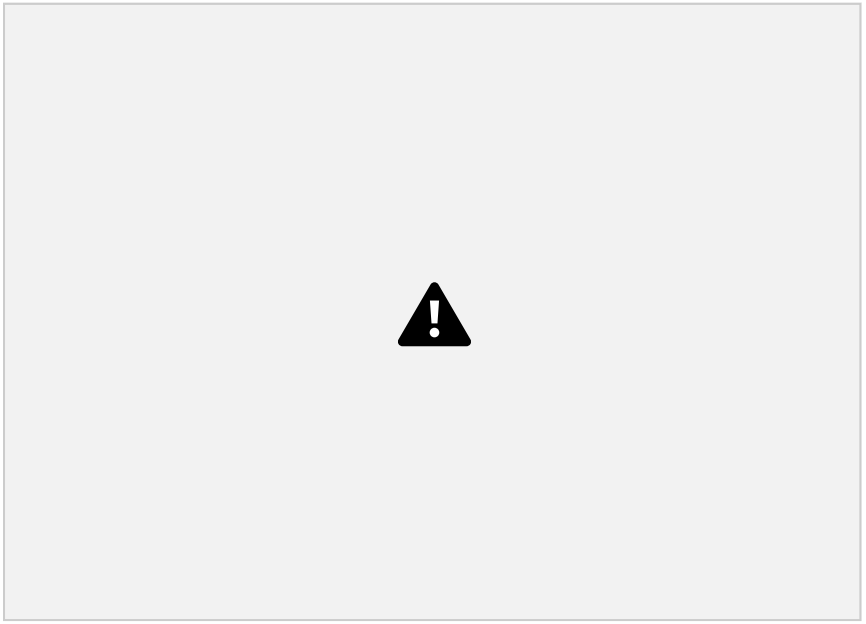
Route::middleware(['auth', 'verified'])->group(function () {
 Route::get('dashboard', DashboardController::class)->name('dashboard');
});
```

```
require __DIR__.'/settings.php';
```

- Route::middleware(['auth', 'verified']) → *dashboard* hanya dapat diakses setelah *login*. Middleware verified tidak menghalangi karena *model* User tidak mengimplementasikan MustVerifyEmail.
- *Route* login, register, dan logout tidak ditulis di sini karena sudah didaftarkan otomatis oleh Fortify, sedangkan settings.php berisi *route* halaman pengaturan profil bawaan

starter kit. Jika file routes/web.php bawaan berisi baris require lain, baris tersebut tetap dipertahankan.

- *Route* milik modul juga tidak ditulis di sini, karena setiap modul mendaftarkan *route* nya sendiri pada file Modules/NamaModul/routes/web.php.



Gambar 9.16 Route dashboard pada routes/web.php

16. Buka file resources/js/pages/dashboard.tsx, lalu ganti isinya dengan kode berikut:

```
import { Head, Link } from '@inertiajs/react';
import { ClipboardList, ShieldCheck, Users } from 'lucide-react';
import { useSharedProps } from '@/hooks/use-shared-props';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';

type DashboardProps = {
 totalPendaftar: number;
 totalUsers: number | null;
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Dashboard', href: '/dashboard' }];

export default function Dashboard({ totalPendaftar, totalUsers }:
DashboardProps) {  const { user, isAdmin } = useSharedProps();

 return (
 <AppLayout breadcrumbs={breadcrumbs}>
 <Head title="Dashboard" />

 <div className="flex flex-1 flex-col gap-4 p-4">
 <div className="rounded-xl border border-sidebar-border/70 p-6
dark:border-sidebar-border">
 <h1 className="text-xl font-semibold">Selamat datang,
{user.name}!</h1>
 <p className="mt-1 flex items-center gap-2 text-sm text-muted foreground">
 <ShieldCheck className="size-4" />
                      Anda login sebagai {isAdmin ? 'Admin' : 'User'}.
 </p>
 </div>

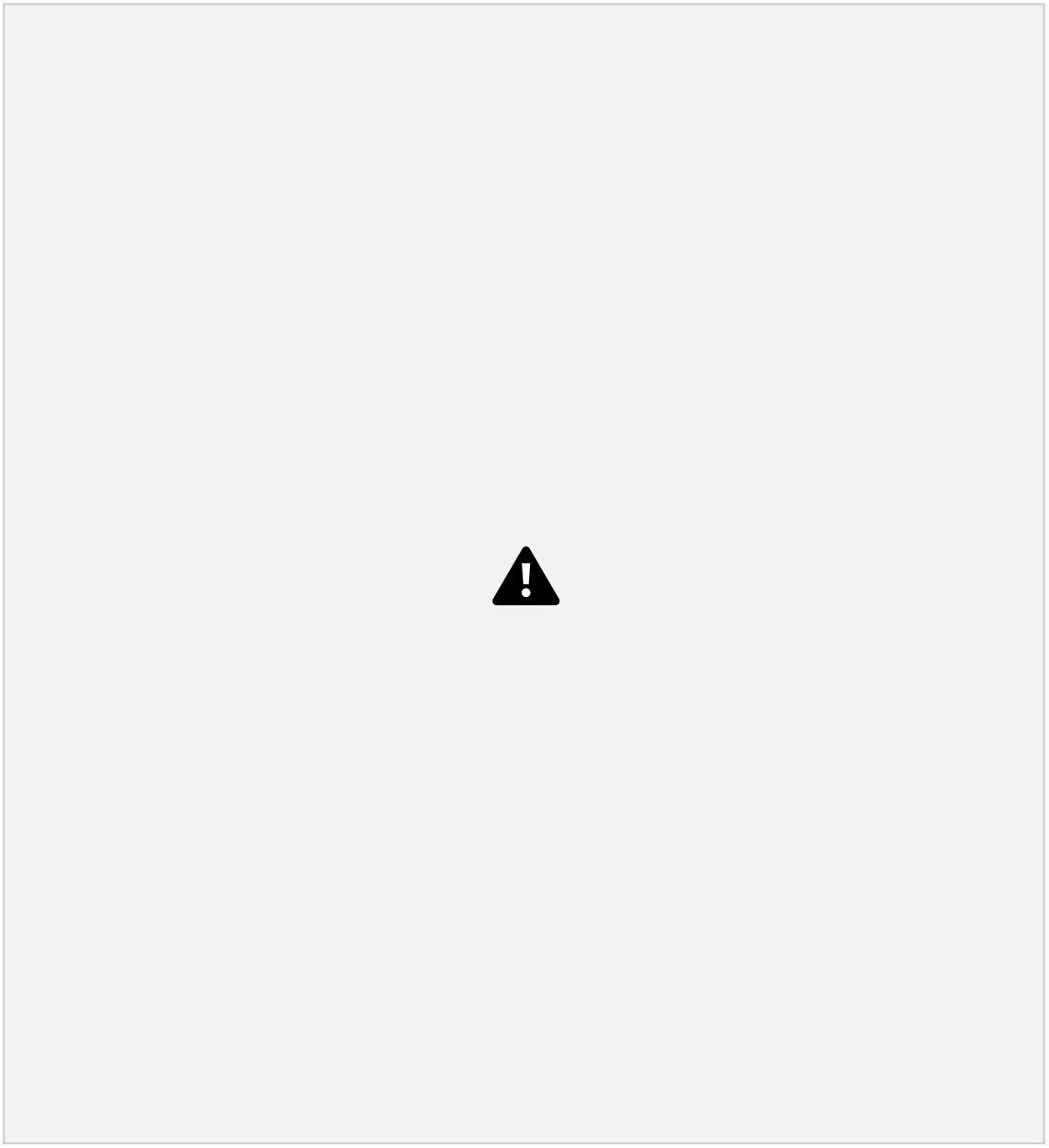
 <div className="grid gap-4 md:grid-cols-2">
```



```
 <Link
 href="/pendaftaran"
                       className="rounded-xl border border-sidebar-border/70 p-6
transition hover:bg-accent dark:border-sidebar-border"
 >
 <ClipboardList className="size-6 text-muted-foreground" />  <p className="mt-4
text-3xl font-bold">{totalPendaftar}</p>  <p className="text-sm
text-muted-foreground">Jumlah  pendaftar</p>
 </Link>

 {isAdmin && totalUsers !== null && (
 <Link
 href="/admin/users"
                         className="rounded-xl border border-sidebar-border/70 p-6
transition hover:bg-accent dark:border-sidebar-border"
 >
 <Users className="size-6 text-muted-foreground" />  <p className="mt-4 text-3xl
font-bold">{totalUsers}</p>  <p className="text-sm text-muted-foreground">Jumlah
user</p>
 </Link>
 )}
 </div>
 </div>
 </AppLayout>
 );
}
```

- AppLayout → *layout* bawaan *starter kit* yang berisi *sidebar*, *header*, dan *breadcrumb*. Seluruh halaman modul juga memakai *layout* ini sehingga tampilannya seragam (pengganti @extends('layouts.app')).
- { totalPendaftar, totalUsers } → *props* yang dikirim oleh DashboardController diterima sebagai parameter komponen.
- Head → mengatur judul tab *browser*, sedangkan breadcrumbs → menampilkan jejak navigasi di bagian atas halaman.



Gambar 9.17 Halaman dashboard.tsx

Tahap C: Authorization dengan Middleware Role

Middleware auth hanya memastikan pengguna sudah *login*. Untuk membedakan hak akses admin dan user, kita membuat *middleware* sendiri yang memeriksa kolom role. Tahap ini sama persis dengan Modul 8 karena *authorization* dilakukan di sisi *server*, tidak bergantung pada tampilan Blade maupun React.

1. Buat *middleware* dengan perintah berikut:

```
php artisan make:middleware EnsureUserHasRole
```

Buka file app/Http/Middleware/EnsureUserHasRole.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Http\Middleware;
```

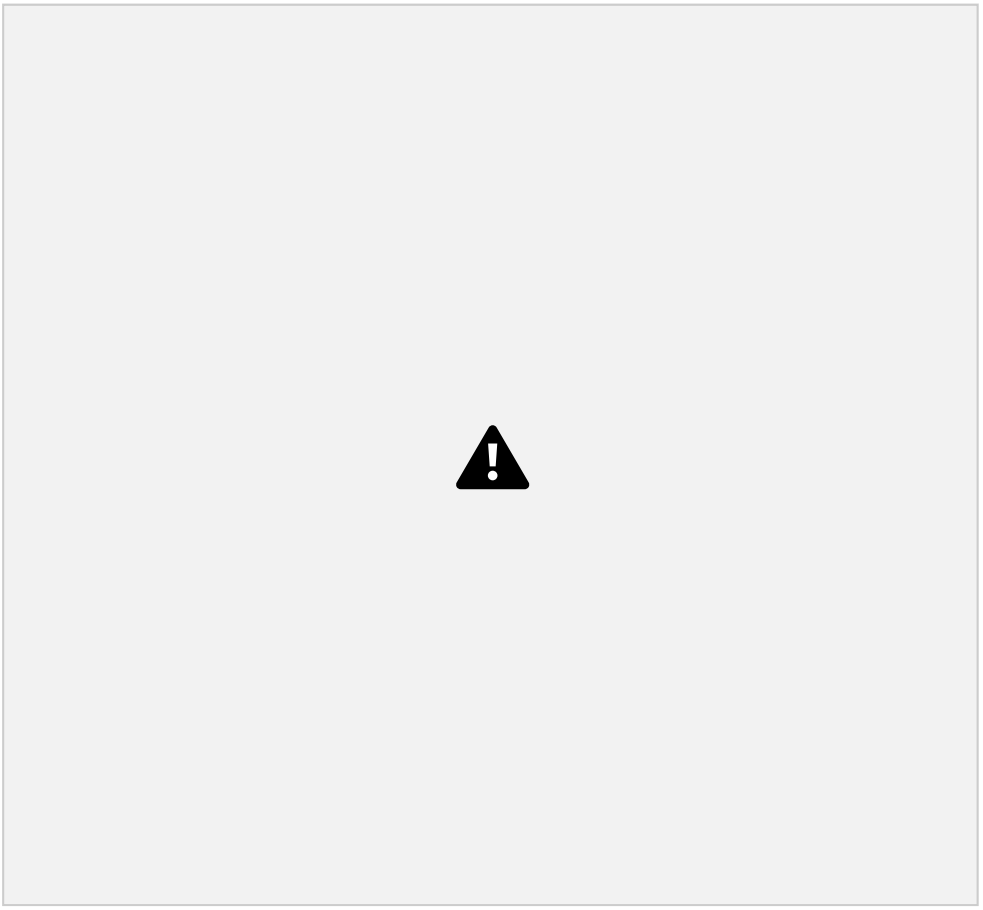
```
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
 /**
 * Ensure the authenticated user has one of the given roles.
 *
 * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
*
 * @param Closure(Request): (Response) $next
 */
 public function handle(Request $request, Closure $next, string ...$roles):
Response
 {
 $user = $request->user();

 if ($user === null || ! $user->hasRole(...$roles)) {
 abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');  }

 return $next($request);
 }
}
```

- string ...$roles → menampung parameter *middleware*. Penulisan role:admin,user menghasilkan $roles = ['admin', 'user'].
- $user->hasRole(...$roles) → memeriksa apakah role pengguna termasuk salah satu role yang diizinkan.
- abort(403, ...) → menghentikan *request* dengan *error* 403 *Forbidden* beserta pesan berbahasa Indonesia.
- $next($request) → meneruskan *request* ke *controller* jika role sesuai.



Gambar 9.18 Middleware EnsureUserHasRole

2. Daftarkan *middleware* dengan nama alias role pada file bootstrap/app.php. Tambahkan baris use di bagian atas dan bagian alias di dalam withMiddleware(); baris bawaan

starter kit (encryptCookies dan web append) dibiarkan seperti semula:

```
use App\Http\Middleware\EnsureUserHasRole;
// ... use lain bawaan starter kit dibiarkan

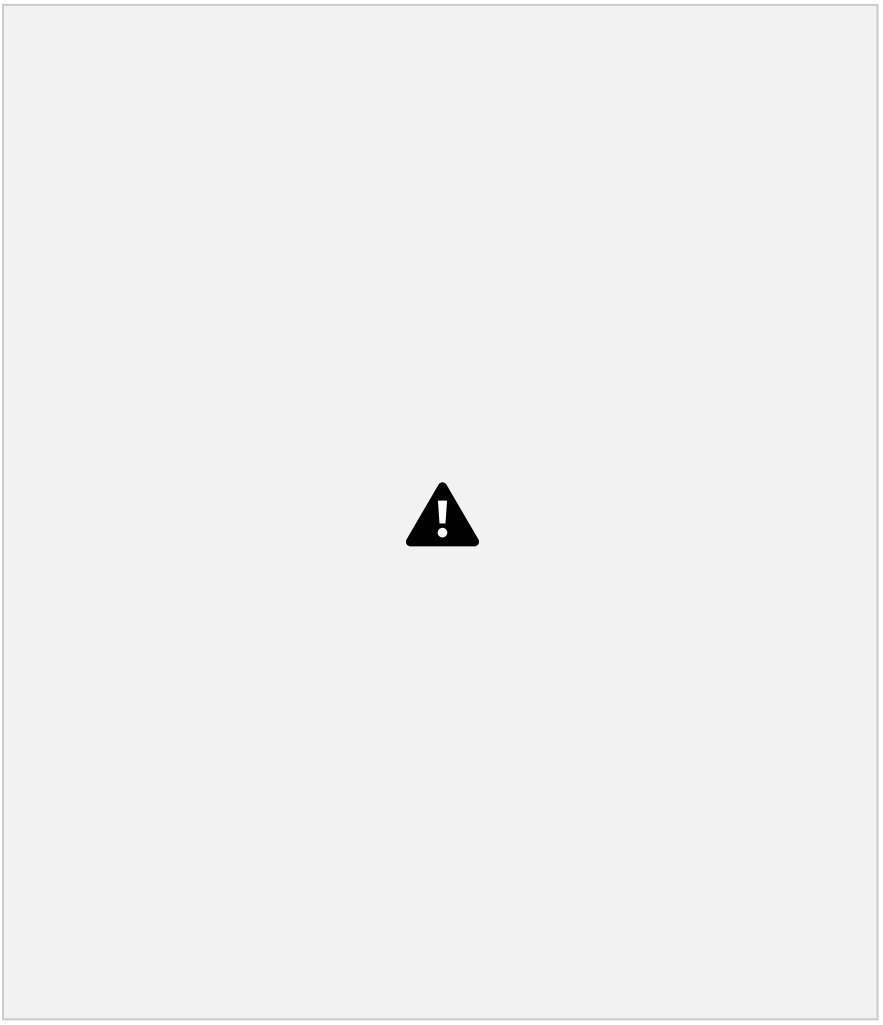
 ->withMiddleware(function (Middleware $middleware): void {
 $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

 $middleware->web(append: [
 HandleAppearance::class,
 HandleInertiaRequests::class,
 AddLinkHeadersForPreloadedAssets::class,
 ]);

 $middleware->alias([
 'role' => EnsureUserHasRole::class,
 ]);
 })
```

- $middleware->alias(['role' => EnsureUserHasRole::class]) → *middleware* dapat dipanggil dengan nama pendek role di *route*. Laravel versi terbaru tidak lagi memiliki file app/Http/Kernel.php, sehingga alias didaftarkan di bootstrap/app.php.
- $middleware->web(append: [...]) → bawaan *starter kit*. HandleInertiaRequests yang berisi *shared props* auth dan flash dijalankan pada setiap *request* web, sehingga baris ini tidak boleh dihapus.

- Berbeda dengan Modul 8, redirectGuestsTo() dan redirectUsersTo() tidak perlu ditulis karena Fortify sudah menyediakan *route* bernama login dan mengarahkan pengguna yang sudah *login* ke /dashboard.



Gambar 9.19 Alias middleware role pada bootstrap/app.php

3. *Middleware* role digunakan pada *route* modul dengan cara berikut (diterapkan pada Tahap D dan E):

```
// Hanya admin
Route::middleware(['auth', 'role:admin'])->group(function () { ... });

// Admin dan user
Route::middleware(['auth', 'role:admin,user'])->group(function () { ... });
```

Urutan *middleware* penting: auth dijalankan lebih dulu agar tamu diarahkan ke *login*, baru kemudian role memeriksa hak akses pengguna yang sudah *login*.

Tahap D: Modul UserManagement (CRUD Inertia Khusus Admin)

Modul ini memungkinkan admin menambah, mengubah, dan menghapus akun pengguna. Setiap aksi dikirim melalui form React menggunakan useForm() milik Inertia. *Controller* tetap bekerja seperti CRUD *synchronous* pada Modul 8, yaitu mengembalikan halaman (Inertia::render()) atau *redirect* beserta pesan *flash*. Perbedaannya, Inertia mengganti halaman tanpa memuat ulang seluruh *browser*.

1. Buat dua Form Request di dalam modul UserManagement:

```
php artisan module:make-request StoreUserRequest UserManagement
php artisan module:make-request UpdateUserRequest UserManagement
```

Buka file Modules/UserManagement/app/Http/Requests/StoreUserRequest.php, lalu ganti isinya dengan kode berikut:

```
<?php

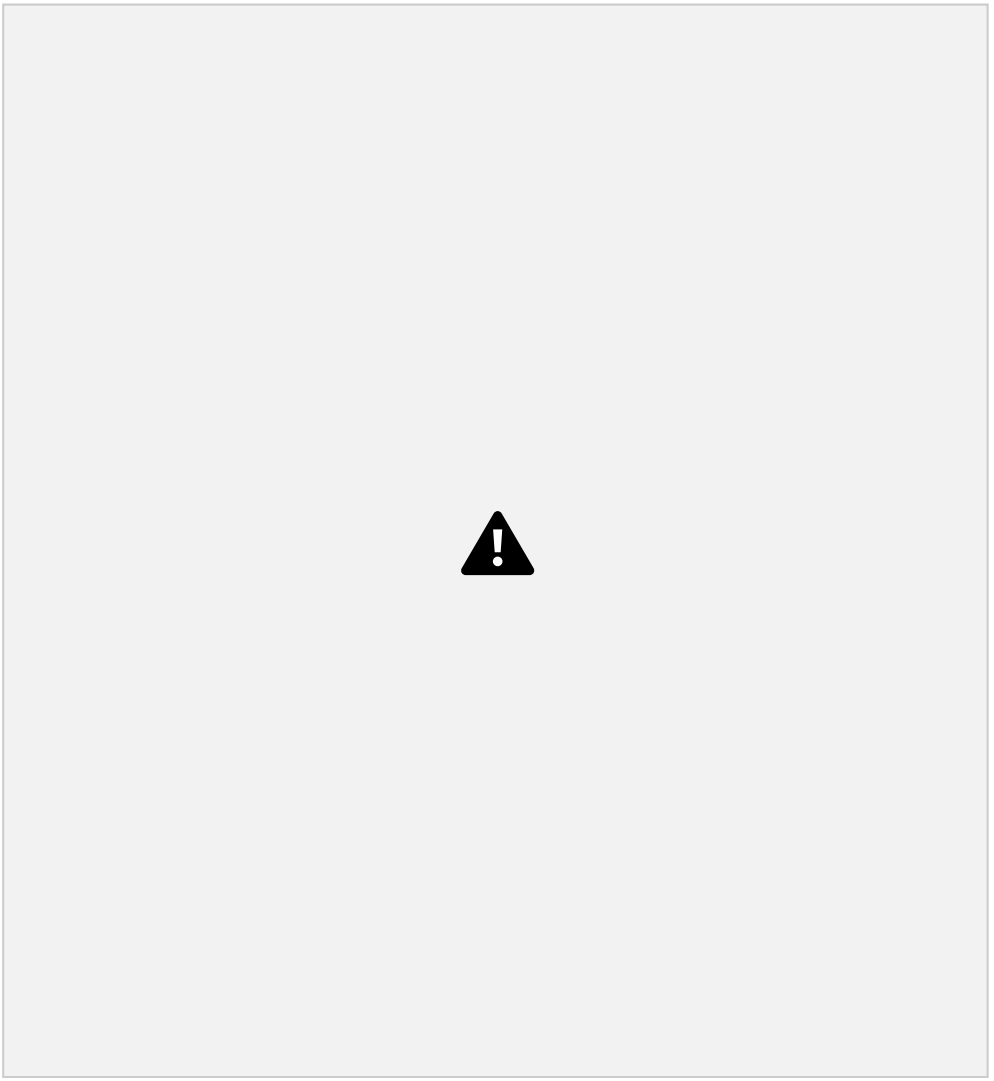
namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
 /**
 * Get the validation rules that apply to the request.
 *
 * @return array<string, array<int, mixed>>
 */
 public function rules(): array
 {
 return [
 'name' => ['required', 'string', 'max:255'],
 'email' => ['required', 'string', 'email', 'max:255',
'unique:users,email'],
 'role' => ['required', Rule::enum(UserRole::class)],
 'password' => ['required', 'confirmed', Password::defaults()],  ];
 }

 /**
 * Determine if the user is authorized to make this request.
 */
 public function authorize(): bool
 {
 return $this->user()?->isAdmin() ?? false;
 }
}
```

- Rule::enum(UserRole::class) → role hanya boleh bernilai admin atau user. Nilai lain seperti superadmin ditolak.
- authorize() → lapisan pengaman kedua: *request* ditolak (403) jika yang mengirim bukan admin, walaupun *route* sudah dilindungi *middleware*.
- Jika validasi gagal, Laravel mengembalikan pengguna ke halaman sebelumnya. Inertia otomatis memasukkan pesan *error* ke objek errors milik useForm(), sehingga pesan dapat ditampilkan di bawah setiap *input*.



Gambar 9.20 StoreUserRequest

Buka file Modules/UserManagement/app/Http/Requests/UpdateUserRequest.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
 /**
 * Get the validation rules that apply to the request.
 *
 * An admin cannot demote their own account, so they never lock themselves out.
*
 * @return array<string, array<int, mixed>>
 */
 public function rules(): array
 {
 $user = $this->route('user');
 $isEditingOwnAccount = $user?->is($this->user()) ?? false;
```



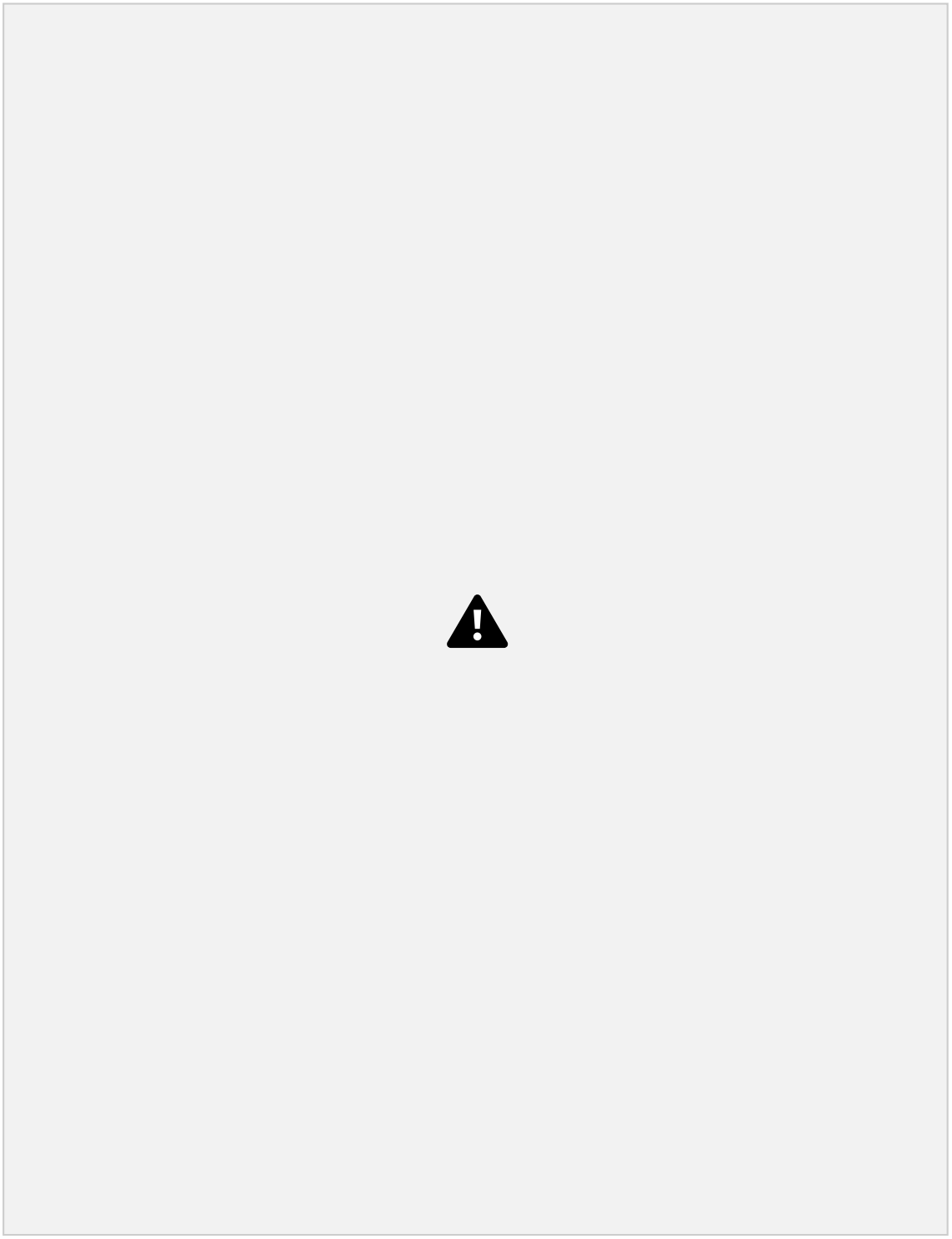
```php
return [
 'name' => ['required', 'string', 'max:255'],
 'email' => ['required', 'string', 'email', 'max:255',
Rule::unique('users', 'email')->ignore($user?->id)],
 'role' => [
 'required',
 Rule::enum(UserRole::class),
 Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),  ],
 'password' => ['nullable', 'confirmed', Password::defaults()],  ];
 }

 /**
 * Get custom messages for validator errors.
 *
 * @return array<string, string>
 */
 public function messages(): array
 {
 return [
 'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',  ];
 }

 /**
 * Determine if the user is authorized to make this request.
 */
 public function authorize(): bool
 {
 return $this->user()?->isAdmin() ?? false;
 }
}
```

- Rule::unique('users', 'email')->ignore($user?->id) → email boleh sama dengan email milik user itu sendiri, tetapi tidak boleh sama dengan user lain.

• Rule::when($isEditingOwnAccount, Rule::in(['admin'])) → admin tidak dapat menurunkan role akunnya sendiri menjadi user, agar tidak terkunci dari menu admin. • 'password' => ['nullable', ...] → *password* boleh dikosongkan ketika edit; *password* lama tetap digunakan.



Gambar 9.21 UpdateUserRequest

2. Buka file Modules/UserManagement/app/Http/Controllers/UserManagementController.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
```

```
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
 /**
 * Display a listing of users, optionally filtered by keyword and role.  */
 public function index(Request $request): Response
 {
 $search = $request->string('search')->trim()->toString();  $role =
UserRole::tryFrom($request->string('role')->toString());

 $users = User::query()
 ->when($search !== '', function (Builder $query) use ($search) {
$query->where(function (Builder $query) use ($search) {
$query->where('name', 'like', "%{$search}%")  ->orWhere('email', 'like',
"%{$search}%");  });
 })
 ->when($role, fn (Builder $query) => $query->where('role', $role))
->orderBy('name')
 ->paginate(10)
 ->withQueryString()
 ->through(fn (User $user) => [
 'id' => $user->id,
 'name' => $user->name,
 'email' => $user->email,
 'role' => $user->role->value,
 'role_label' => $user->role->label(),
 'created_at' => $user->created_at?->diffForHumans(),  ]);

 return Inertia::render('UserManagement::index', [
 'users' => $users,
 'filters' => [
 'search' => $search,
 'role' => $role?->value ?? '',
 ],
 'roles' => $this->roleOptions(),
 ]);
 }

 /**
 * Show the form for creating a new user.
 */
 public function create(): Response
 {
 return Inertia::render('UserManagement::create', [
 'roles' => $this->roleOptions(),
 ]);
 }

 /**
 * Store a newly created user.
 */
 public function store(StoreUserRequest $request): RedirectResponse  {
 User::create($request->validated());
```



```
return redirect()->route('admin.users.index')->with('success', 'User berhasil
ditambahkan.');
```

```
 }

 /**
 * Show the form for editing the specified user.
 */
 public function edit(User $user): Response
 {
 return Inertia::render('UserManagement::edit', [
 'user' => [
 'id' => $user->id,
 'name' => $user->name,
 'email' => $user->email,
 'role' => $user->role->value,
 ],
 'roles' => $this->roleOptions(),
 ]);
 }

 /**
 * Update the specified user. The password is only changed when filled in.  */
 public function update(UpdateUserRequest $request, User $user): RedirectResponse  {
 $attributes = $request->validated();

 if (blank($attributes['password'] ?? null)) {
 unset($attributes['password']);
 }

 $user->update($attributes);

 return redirect()->route('admin.users.index')->with('success', 'User berhasil
diperbarui.');
 }

 /**
 * Remove the specified user. An admin cannot delete their own account.  */
 public function destroy(Request $request, User $user): RedirectResponse  {
 if ($user->is($request->user())) {
 return redirect()->route('admin.users.index')->with('error', 'Anda tidak  dapat
menghapus akun Anda sendiri.');
 }

 $user->delete();

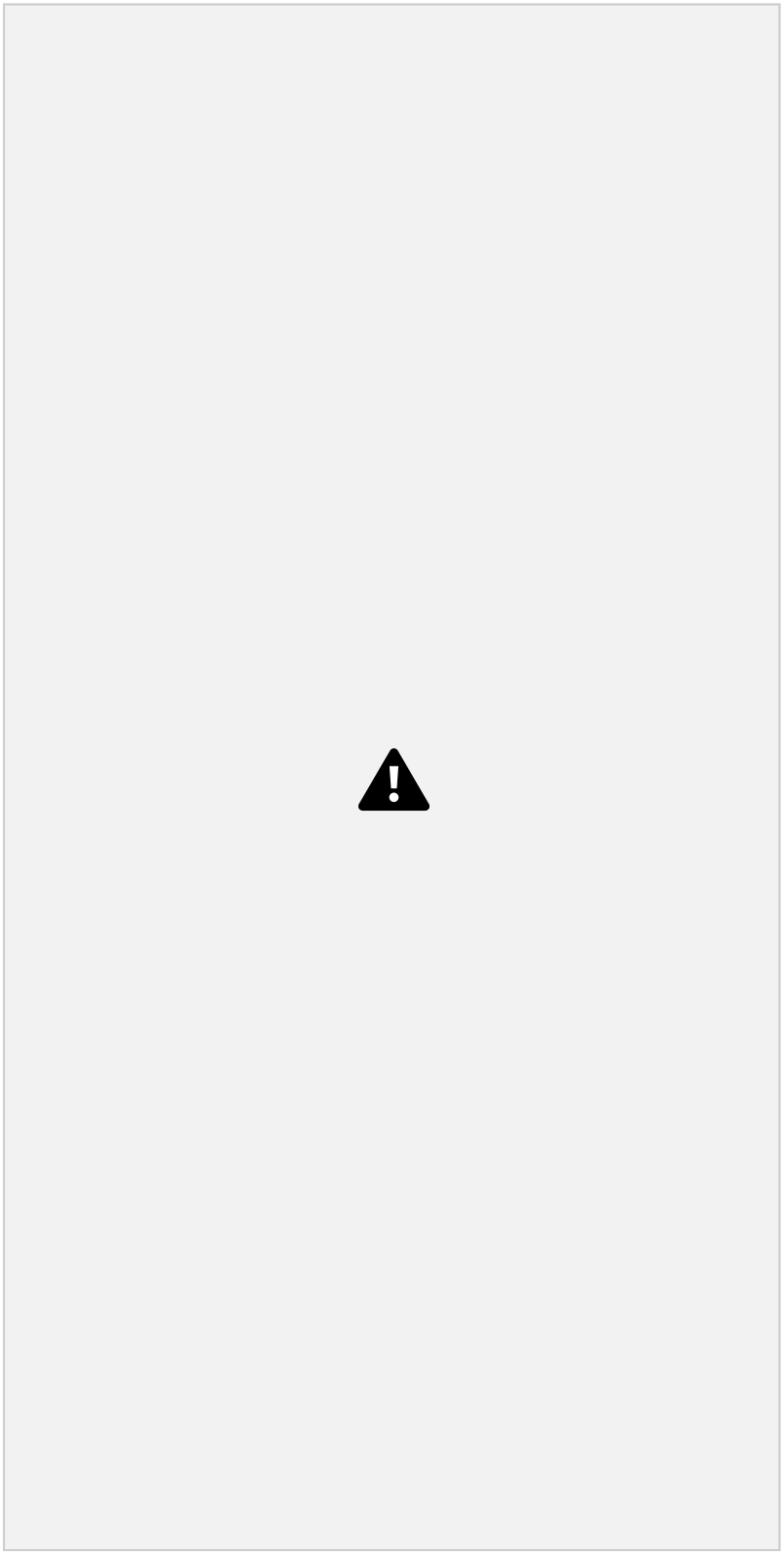
 return redirect()->route('admin.users.index')->with('success', 'User berhasil
dihapus.');
 }

 /**
 * Role options for the select input on the React page.
 *
 * @return list<array{value: string, label: string}>
 */
 private function roleOptions(): array
 {
 return array_map(
 fn (UserRole $role) => ['value' => $role->value, 'label' => $role- >label()],
 UserRole::cases(),
```

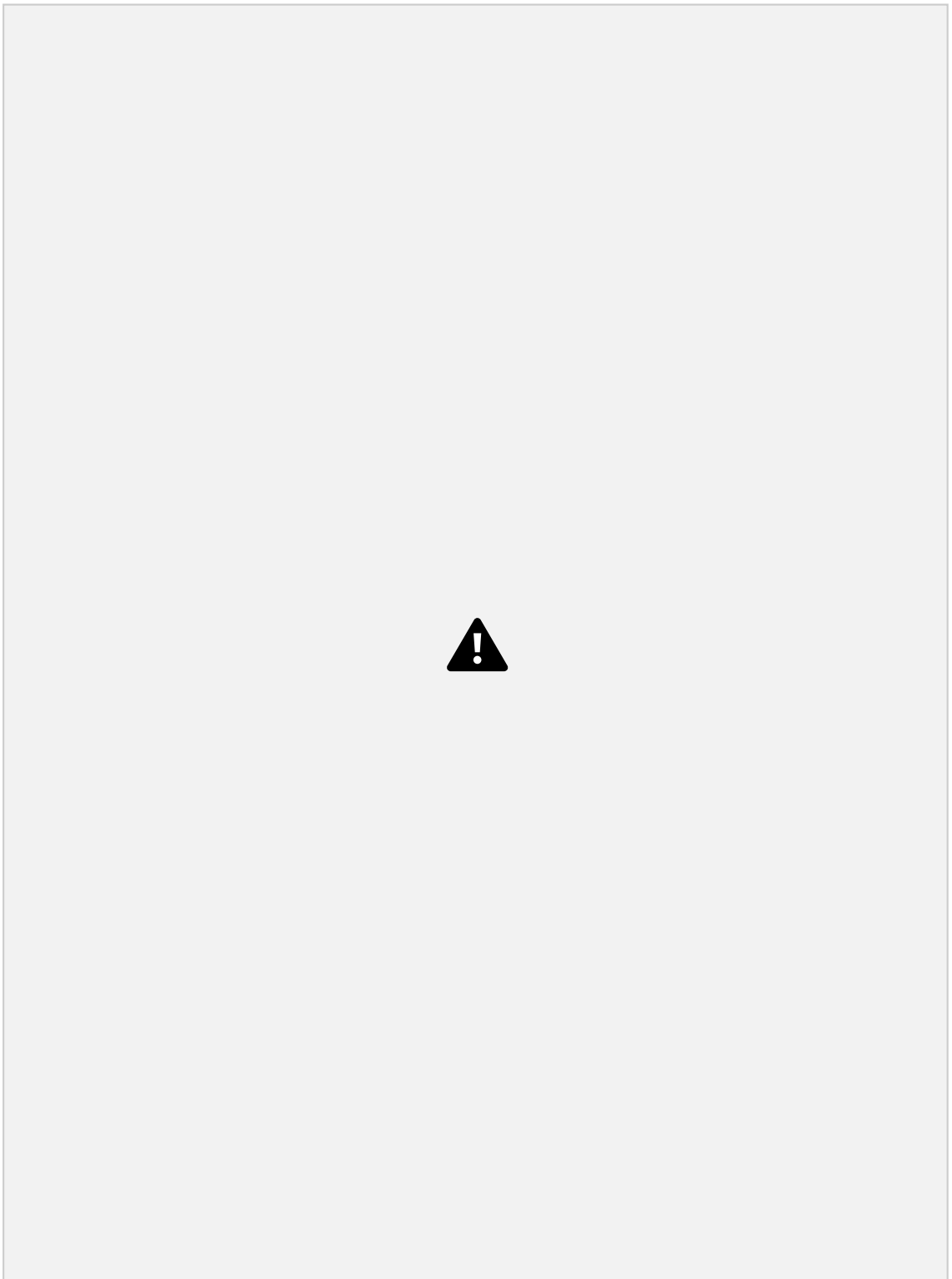


```
 );
  }
 }
```

- index() → menampilkan daftar user dengan pencarian (*search*) berdasarkan nama atau email, filter role, dan *pagination* 10 data per halaman. withQueryString() menjaga kata kunci dan filter tetap terbawa ketika berpindah halaman *pagination*.
- through(fn (User $user) => [...]) → mengubah setiap data user menjadi *array* berisi kolom yang dibutuhkan halaman saja. Data sensitif seperti *password* dan *remember_token* tidak ikut dikirim ke *browser*.
- Inertia::render('UserManagement::index', [...]) → memanggil halaman React dari dalam modul menggunakan format NamaModul::halaman (pengganti view('usermanagement::index') pada Modul 8).
- store() dan update() → hanya menyimpan data hasil validasi ($request->validated()). Pada update, *password* dihapus dari data jika dikosongkan. Keduanya kembali ke daftar user dengan pesan *flash* yang ditampilkan oleh komponen FlashMessage.
- destroy() → admin tidak dapat menghapus akunnya sendiri; aplikasi kembali ke daftar user dengan pesan *error*.
- roleOptions() → menyiapkan pilihan role berupa pasangan value dan label untuk elemen select pada form React.
- User $user → *route model binding*; Laravel otomatis mengambil data user berdasarkan id pada URL dan menampilkan 404 jika tidak ditemukan.



Gambar 9.22 UserManagementController bagian 1 (index, create, store)



Gambar 9.23 UserManagementController bagian 2 (edit, update, destroy)

3. Buka file Modules/UserManagement/routes/web.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Support\Facades\Route;
use

Modules\UserManagement\Http\Controllers\UserManagementController;

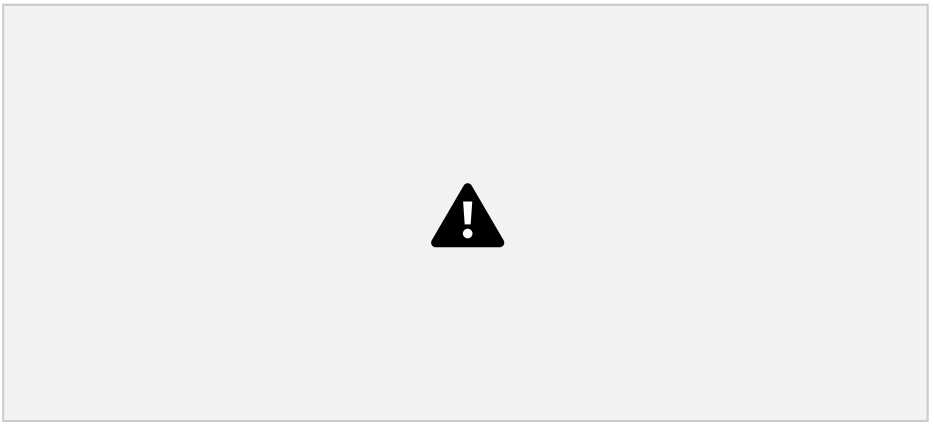
Route::middleware(['auth', 'role:admin'])
```



```
->prefix('admin')
 ->name('admin.')
 ->group(function () {
 Route::resource('users', UserManagementController::class)->except('show');  });
```

• middleware(['auth', 'role:admin']) → seluruh *route* modul ini hanya untuk admin. • prefix('admin') dan name('admin.') → URL menjadi /admin/users dan nama *route* menjadi admin.users.index, admin.users.create, dan seterusnya.

• Route::resource(...)->except('show') → membuat enam *route* CRUD sekaligus tanpa halaman detail.



Gambar 9.24 Route modul UserManagement

4. Buat file Modules/UserManagement/resources/js/pages/index.tsx sebagai halaman daftar user:

```
import { Head, Link, router } from '@inertiajs/react';
import { Pencil, Plus, Search, Trash2 } from 'lucide-react';
import { useState } from 'react';
import type { FormEvent } from 'react';
import { FlashMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { useSharedProps } from '@/hooks/use-shared-props';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';

type UserRow = {
 id: number;
 name: string;
 email: string;
 role: 'admin' | 'user';
 role_label: string;
 created_at: string | null;
};

type PaginationLink = { url: string | null; label: string; active: boolean };

type IndexProps = {
 users: {
 data: UserRow[];
 links: PaginationLink[];
 from: number | null;
 to: number | null;
```



```
total: number;
 };
 filters: { search: string; role: string };
 roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Kelola User', href: '/admin/users'
}];

export default function Index({ users, filters, roles }: IndexProps)
{  const { user: currentUser } = useSharedProps();
 const [search, setSearch] = useState(filters.search);
 const [deleting, setDeleting] = useState<UserRow | null>(null);
const [processing, setProcessing] = useState(false);

 // Kirim ulang request GET /admin/users dengan kata kunci dan filter role
const applyFilters = (params: { search?: string; role?: string }) => {
router.get('/admin/users', { ...filters, search, ...params }, {
preserveState: true, replace: true });
 };

 const handleSearch = (event: FormEvent) => {
 event.preventDefault();
 applyFilters({ search });
 };

 const handleDelete = () => {
 if (!deleting) {
 return;
 }

 router.delete(`/admin/users/${deleting.id}`, {
 preserveScroll: true,
 onStart: () => setProcessing(true),
```

```
 onFinish: () => {
 setProcessing(false);
 setDeleting(null);
 },
 });
 };

 return (
 <AppLayout breadcrumbs={breadcrumbs}>
 <Head title="Kelola User" />

 <div className="flex flex-1 flex-col gap-4 p-4">
 <div className="flex flex-wrap items-center justify-between gap-2">  <div>
 <h1 className="text-xl font-semibold">Kelola User</h1>  <p className="text-sm
text-muted-foreground">Total  {users.total} user terdaftar.</p>
 </div>
                    <Button asChild>
 <Link href="/admin/users/create">
 <Plus /> Tambah User
 </Link>
 </Button>
 </div>

 <FlashMessage />

 <div className="flex flex-wrap gap-2">
 <form onSubmit={handleSearch} className="flex flex-1 gap-2">  <Input
 value={search}
```



```
   onChange={(event) => setSearch(event.target.value)} placeholder="Cari nama
                                atau email..."
                            className="max-w-sm"
 />
                        <Button type="submit" variant="secondary">
 <Search /> Cari
 </Button>
 </form>
                    <select
 value={filters.role}
                            onChange={(event) => applyFilters({ role: event.target.value
})}
 className="h-9 rounded-md border border-input bg-transparent  px-3 text-sm
shadow-xs"
 >
 <option value="">Semua role</option>
                        {roles.map((role) => (
 <option key={role.value} value={role.value}>  {role.label}
 </option>
 ))}
 </select>
 </div>

 <div className="overflow-x-auto rounded-xl border">  <table
className="w-full text-sm">
 <thead className="bg-muted/50 text-left">  <tr>
 <th className="px-4 py-3 font-medium">No</th>  <th className="px-4 py-3
font-medium">Nama</th>  <th className="px-4 py-3 font-medium">Email</th>  <th
className="px-4 py-3 font-medium">Role</th>  <th className="px-4 py-3
font-medium">Dibuat</th>  <th className="px-4 py-3 text-right font
medium">Aksi</th>
 </tr>
 </thead>
                        <tbody>
 {users.data.length === 0 && (
 <tr>
```

```
<td colSpan={6} className="px-4 py-8 text-center  text-muted-foreground">
Data user tidak ditemukan.  </td>
</tr>
)}
                            {users.data.map((user, index) => (
<tr key={user.id} className="border-t">  <td className="px-4 py-3">{(users.from ??
1) +  index}</td>
<td className="px-4 py-3 font-medium">  {user.name}
                                        {user.id === currentUser.id && (
<Badge variant="outline" className="ml 2">
Anda
</Badge>
)}
</td>
                                    <td className="px-4 py-3">{user.email}</td>
<td className="px-4 py-3">
<Badge variant={user.role === 'admin' ?  'default' :
'secondary'}>{user.role_label}</Badge>
</td>
```



```
<td className="px-4 py-3 text-muted foreground">{user.created_at}</td>
<td className="px-4 py-3">
<div className="flex justify-end gap-2">  <Button asChild size="sm"
variant="outline">
<Link
href={`/admin/users/${user.id}/edit`}>
<Pencil /> Edit  </Link>
</Button>
                                            {user.id !== currentUser.id && (
<Button size="sm"  variant="destructive" onClick={() =>
setDeleting(user)}>
<Trash2 /> Hapus  </Button>
)}
</div>
</td>
</tr>
))}
</tbody>
</table>
</div>

<div className="flex flex-wrap items-center justify-between gap-2  text-sm">
<p className="text-muted-foreground">
Menampilkan {users.from ?? 0}-{users.to ?? 0} dari  {users.total} data
</p>
                    <div className="flex flex-wrap gap-1">
{users.links.map((link, index) =>
link.url ? (
<Button key={index} asChild size="sm"  variant={link.active ?
'default' : 'outline'}>
<Link href={link.url} preserveScroll>  {cleanLabel(link.label)}  </Link>
</Button>
) : (
<Button key={index} size="sm" variant="outline"  disabled>
{cleanLabel(link.label)}
</Button>
),
)}
</div>
</div>
</div>

<ConfirmDialog
open={deleting !== null}
```

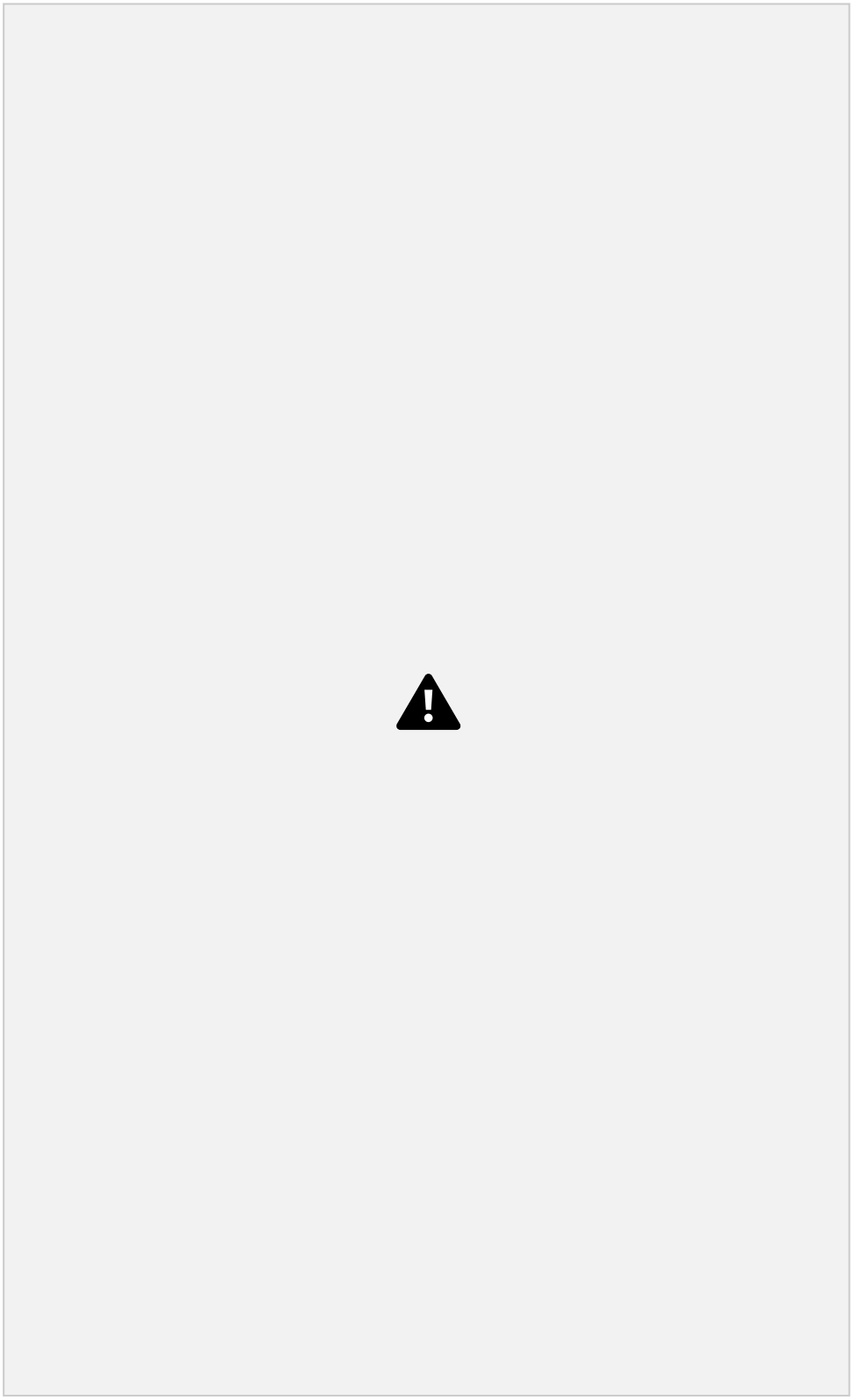
```
 description={`Yakin ingin menghapus user ${deleting?.name}? Tindakan  ini tidak
dapat dibatalkan.`}
 processing={processing}
 onConfirm={handleDelete}
 onOpenChange={(open) => !open && setDeleting(null)}  />
 </AppLayout>
 );
}

// Label pagination Laravel berisi entitas HTML &laquo; dan
&raquo; function cleanLabel(label: string): string {
```



```
 return label.replace('&laquo;', '«').replace('&raquo;', '»');
 }
```

- export default function Index({ users, filters, roles }) → *props* dari Inertia::render('UserManagement::index', [...]) diterima sebagai parameter komponen. Halaman tetap memakai AppLayout milik aplikasi utama sehingga tampilan seluruh modul seragam.
- router.get('/admin/users', {...}, { preserveState: true }) → pencarian dan filter role dikirim sebagai *query string* ke method index(). preserveState menjaga isi kotak pencarian, sedangkan onChange pada select membuat daftar langsung diperbarui ketika role dipilih (pengganti onchange="this.form.submit()").
- (users.from ?? 1) + index → nomor urut tetap berlanjut pada halaman *pagination* berikutnya.
- users.links → tautan *pagination* dari *paginator* Laravel. Setiap tautan dirender dengan komponen Link sehingga perpindahan halaman tidak memuat ulang *browser*.
- setDeleting(user) → menyimpan user yang akan dihapus lalu membuka ConfirmDialog. *Request* router.delete() baru dikirim setelah pengguna menekan Ya, hapus. Tombol Hapus tidak ditampilkan pada baris akun yang sedang *login*.



Gambar 9.25 Tabel daftar user pada halaman index.tsx

5. Buat *folder* Modules/UserManagement/resources/js/components, lalu buat file user form.tsx yang dipakai bersama oleh form tambah dan edit:

```
import { Link } from '@inertiajs/react';
import type { FormEvent } from 'react';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
```

```
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export type UserFormData = {
 name: string;
 email: string;
 role: string;
 password: string;
 password_confirmation: string;
};

type UserFormProps = {
 data: UserFormData;
 errors: Partial<Record<keyof UserFormData, string>>;
 processing: boolean;
 roles: { value: string; label: string }[];
 isEdit?: boolean;
 submitLabel: string;
 setData: (key: keyof UserFormData, value: string) => void;
 onSubmit: (event: FormEvent) => void;
};

export function UserForm({ data, errors, processing, roles, isEdit = false,
submitLabel, setData, onSubmit }: UserFormProps) {
 return (
 <form onSubmit={onSubmit} noValidate className="max-w-xl space-y-5 rounded-xl  border
p-6">
 <div className="grid gap-2">
 <Label htmlFor="name">
 Nama <span className="text-red-500">*</span>
 </Label>
 <Input id="name" value={data.name} onChange={(event) =>
setData('name', event.target.value)} autoFocus />
 <InputError message={errors.name} />
 </div>

 <div className="grid gap-2">
 <Label htmlFor="email">
 Email <span className="text-red-500">*</span>  </Label>
 <Input id="email" type="email" value={data.email} onChange={(event)  =>
setData('email', event.target.value)} />
 <InputError message={errors.email} />
 </div>

 <div className="grid gap-2">
 <Label htmlFor="role">
 Role <span className="text-red-500">*</span>
 </Label>
 <select
 id="role"
                   value={data.role}
                   onChange={(event) => setData('role', event.target.value)}
                   className="h-9 rounded-md border border-input bg-transparent px-3
text-sm shadow-xs"
 >
 {roles.map((role) => (
 <option key={role.value} value={role.value}>  {role.label}
 </option>
 ))}
 </select>
 <InputError message={errors.role} />
 </div>
```

```
<div className="grid gap-2">
 <Label htmlFor="password">
 Password {!isEdit && <span className="text-red-500">*</span>}  </Label>
 <Input id="password" type="password" value={data.password}
onChange={(event) => setData('password', event.target.value)} />
 {isEdit && <p className="text-xs text-muted-foreground">Kosongkan  jika tidak
ingin mengganti password.</p>}
 <InputError message={errors.password} />
 </div>

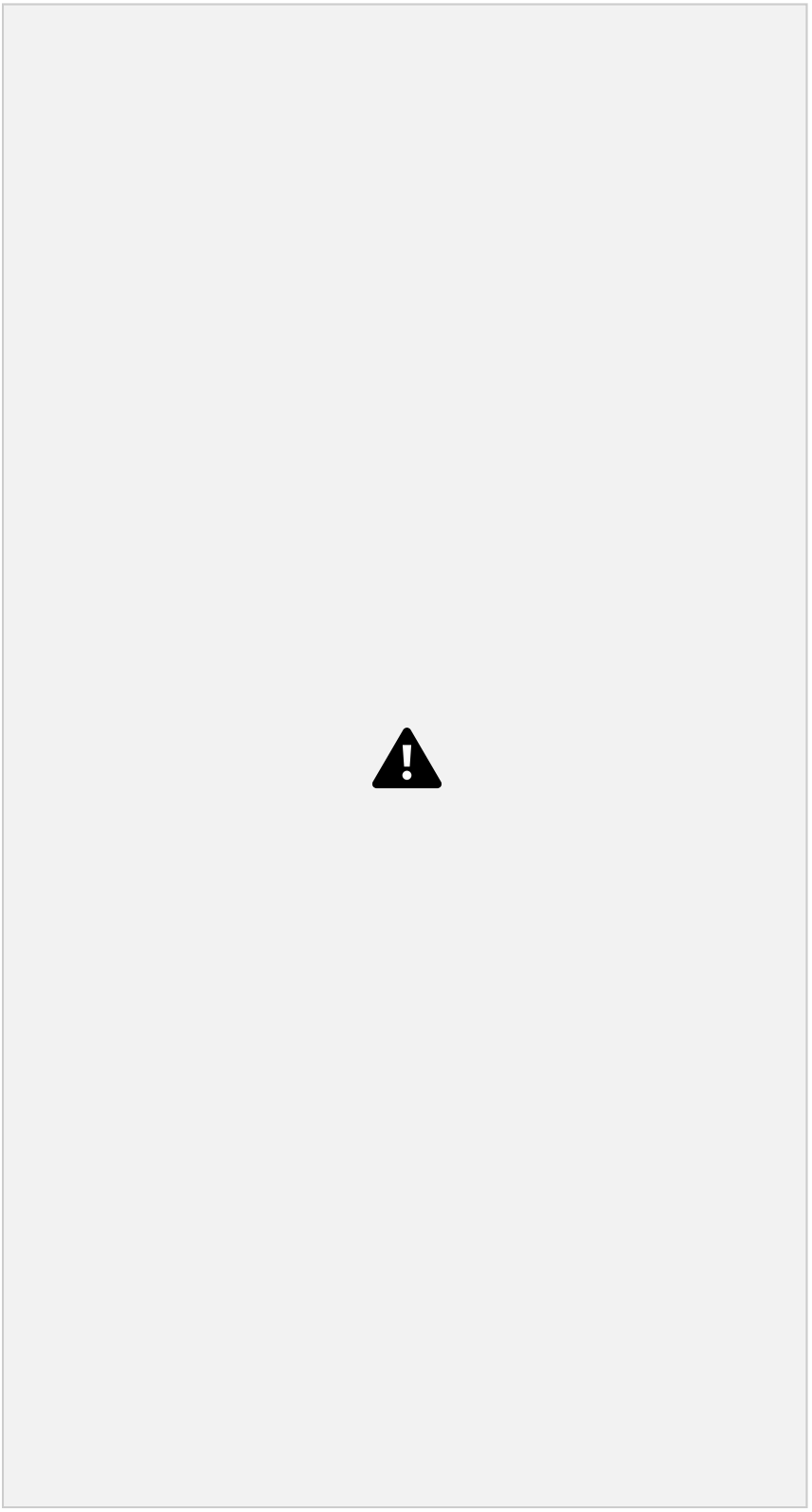
 <div className="grid gap-2">
 <Label htmlFor="password_confirmation">Konfirmasi Password</Label>  <Input
 id="password_confirmation"
                    type="password"
                    value={data.password_confirmation}
                   onChange={(event) => setData('password_confirmation',
event.target.value)}
 />
 </div>

 <div className="flex gap-2">
 <Button type="submit" disabled={processing}>
 {processing ? 'Menyimpan...' : submitLabel}
 </Button>
 <Button asChild variant="secondary">
 <Link href="/admin/users">Batal</Link>
 </Button>
 </div>
 </form>
 );
}
```

- UserForm → komponen form yang dipakai bersama oleh halaman create dan edit (pengganti partial partials/form.blade.php). Komponen ini hanya menampilkan *input*, sedangkan data dan proses pengiriman diatur oleh halaman yang memakainya.
- value={data.name} dan setData('name', ...) → isi *input* disimpan pada *state* useForm(). Pada form edit, *state* sudah terisi data lama sehingga *input* langsung menampilkan nilai sebelumnya (pengganti old('name', $user?->name)).

• InputError message={errors.name} → komponen bawaan *starter kit* yang menampilkan pesan *error* berwarna merah di bawah *input* (pengganti @error pada Blade). • noValidate → validasi bawaan *browser* dimatikan agar seluruh pesan *error* berasal dari Form Request di sisi *server*.

- isEdit → tanda bintang pada *password* hanya ditampilkan saat menambah user, karena *password* opsional saat edit.



Gambar 9.26 Komponen form user (user-form.tsx)

6. Buat file Modules/UserManagement/resources/js/pages/create.tsx:

```
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
```

```
import type { UserFormData } from '../components/user-form';

type CreateProps = {
 roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [
 { title: 'Kelola User', href: '/admin/users' },
 { title: 'Tambah User', href: '/admin/users/create' },
];

export default function Create({ roles }: CreateProps) {
 const { data, setData, post, processing, errors } = useForm<UserFormData>({
name: '',
 email: '',
 role: 'user',
 password: '',
 password_confirmation: '',
 });

 const submit = (event: FormEvent) => {
 event.preventDefault();
 post('/admin/users');
 };

 return (
 <AppLayout breadcrumbs={breadcrumbs}>
 <Head title="Tambah User" />

 <div className="flex flex-1 flex-col gap-4 p-4">
 <div>
 <h1 className="text-xl font-semibold">Tambah User</h1>  <p className="text-sm
text-muted-foreground">Isi data akun baru  beserta role-nya.</p>
 </div>

 <UserForm
 data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    submitLabel="Simpan"
                    setData={setData}
                    onSubmit={submit}
 />
 </div>
 </AppLayout>
 );
}
```

- useForm({...}) → nilai awal form tambah dibuat kosong dengan role bawaan user.
- post('/admin/users') → Inertia mengirim isi form ke method store(). Token CSRF ditangani otomatis oleh Inertia, sehingga tidak perlu menulis @csrf seperti pada Blade. Jika validasi gagal, halaman tetap terbuka dan errors terisi; jika berhasil, Inertia mengikuti *redirect* ke daftar user.
- import { UserForm } from '../components/user-form' → komponen milik modul di *import* dengan *path* relatif, sedangkan komponen bersama milik aplikasi utama di *import* dengan alias @/.

7. Buat file Modules/UserManagement/resources/js/pages/edit.tsx:

```
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';

type EditProps = {
 user: { id: number; name: string; email: string; role: string };
roles: { value: string; label: string }[];
};

export default function Edit({ user, roles }: EditProps) {
 const breadcrumbs: BreadcrumbItem[] = [
 { title: 'Kelola User', href: '/admin/users' },
 { title: 'Edit User', href: `/admin/users/${user.id}/edit` },  ];

 const { data, setData, put, processing, errors } = useForm<UserFormData>({
name: user.name,
 email: user.email,
 role: user.role,
 password: '',
 password_confirmation: '',
 });

 const submit = (event: FormEvent) => {
 event.preventDefault();
 put(`/admin/users/${user.id}`);
 };

 return (
 <AppLayout breadcrumbs={breadcrumbs}>
 <Head title="Edit User" />

 <div className="flex flex-1 flex-col gap-4 p-4">
 <div>
 <h1 className="text-xl font-semibold">Edit User</h1>  <p className="text-sm
text-muted-foreground">Perbarui data akun  {user.name}.</p>
 </div>

 <UserForm
 data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    isEdit
                    submitLabel="Simpan Perubahan"
                    setData={setData}
                    onSubmit={submit}
 />
 </div>
 </AppLayout>
 );
}
```

- useForm({ name: user.name, ... }) → form edit diisi dengan data lama yang dikirim oleh method edit(), sedangkan *password* dibiarkan kosong.

• put(`/admin/users/${user.id}`) → form dikirim sebagai *request* PUT sehingga diproses oleh method update() (pengganti @method('PUT') pada Blade).

Tahap E: Modul Pendaftaran (CRUD AJAX)

Modul Pendaftaran mengelola data calon mahasiswa tanpa berpindah halaman. Halaman React hanya dimuat satu kali melalui Inertia, kemudian seluruh operasi tampil, tambah, ubah, dan hapus dikirim dengan fetch(). *Controller* tidak mengembalikan halaman atau *redirect*, melainkan data JSON yang dipakai React untuk memperbarui tabel. Kode *backend* pada tahap ini sama dengan Modul 8; perbedaannya terletak pada method index() dan halaman React.

1. Buat *model* Daftar beserta *migration* di dalam modul Pendaftaran:

```
php artisan module:make-model Daftar Pendaftaran --migration
```

Buka file Modules/Pendaftaran/database/migrations/xxxx_xx_xx_xxxxxx_create_daftars_table.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

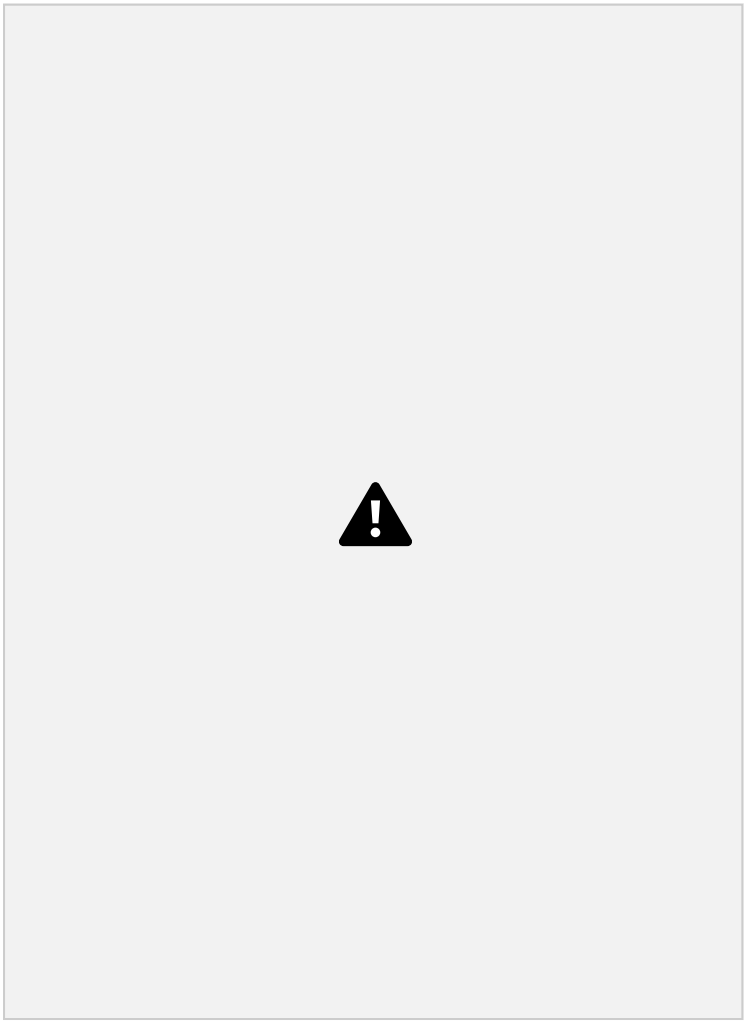
return new class extends Migration
{
 /**
 * Run the migrations.
 */
 public function up(): void
 {
 Schema::create('daftars', function (Blueprint $table) {
 $table->id();
 $table->string('nama');
 $table->string('asal_sekolah');
 $table->string('prodi_tujuan');
 $table->string('email')->unique();
 $table->string('no_hp', 20);
 $table->timestamps();
 });
 }

 /**
 * Reverse the migrations.
 */
 public function down(): void
 {
 Schema::dropIfExists('daftars');
 }
};
```

• Tabel daftars menyimpan lima item data: nama, asal_sekolah, prodi_tujuan, email

(unik), dan no_hp.

- string('no_hp', 20) → nomor HP disimpan sebagai teks agar angka 0 di depan tidak hilang.



Gambar 9.27 Migration tabel daftars

2. Jalankan *migration*. *Migration* milik modul ikut dijalankan oleh perintah migrate biasa:

```
php artisan migrate

# atau khusus satu modul
php artisan module:migrate Pendaftaran
```

3. Tambahkan kolom untuk menyimpan berkas dengan membuat *migration* baru di dalam modul Pendaftaran:

```
php artisan module:make-migration add_berkas_to_daftars_table Pendaftaran
```

Buka file Modules/Pendaftaran/database/migrations/xxxx_xx_xx_xxxxxx_add_berkas_to_daftars_table.p hp, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

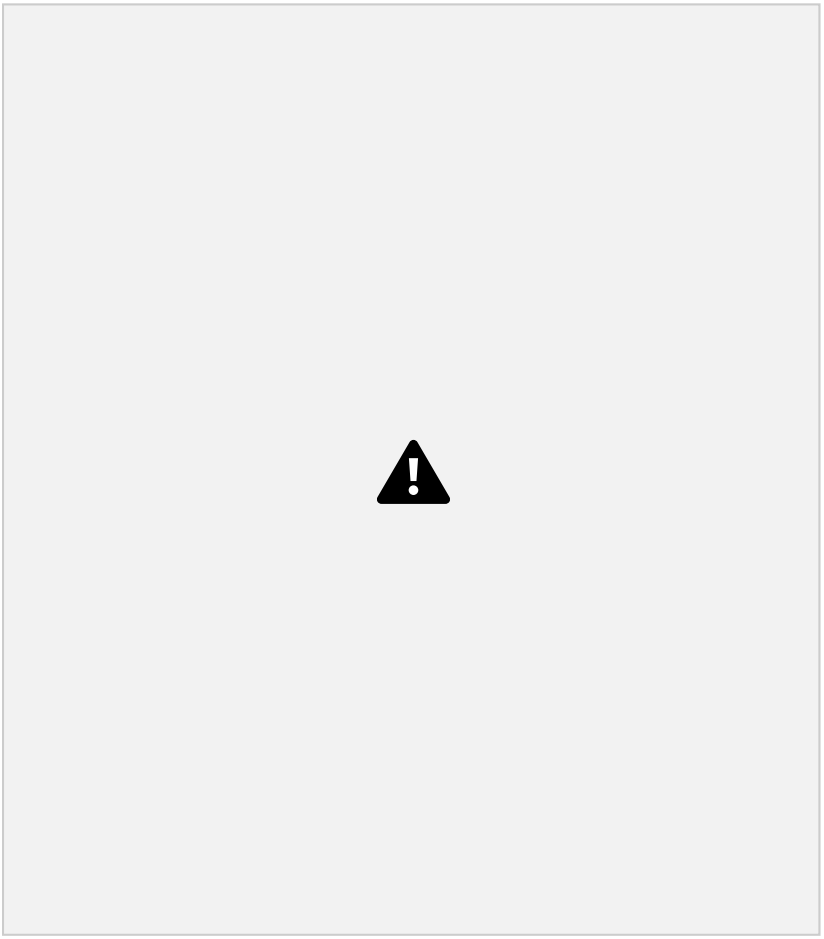
return new class extends Migration
{
 /**
 * Run the migrations.
 */
```



```
public function up(): void
 {
 Schema::table('daftars', function (Blueprint $table) {
 $table->string('berkas')->nullable()->after('no_hp');
 $table->string('nama_berkas')->nullable()->after('berkas');  });
 }

 /**
 * Reverse the migrations.
 */
 public function down(): void
 {
 Schema::table('daftars', function (Blueprint $table) {
 $table->dropColumn(['berkas', 'nama_berkas']);
 });
 }
};
```

- berkas → menyimpan *path* berkas hasil store(), misalnya berkas_pendaftaran/Xy12AbCd.pdf. Isi file tetap disimpan di *folder storage*, bukan di *database*.
- nama_berkas → menyimpan nama asli file (misalnya rapor_budi.png) agar mudah dikenali saat ditampilkan pada tabel.
- nullable() → kolom boleh kosong, sehingga data yang dibuat sebelum fitur *upload* ditambahkan tetap valid.



Gambar 9.28 Migration penambahan kolom berkas

4. Jalankan *migration*, lalu buat *symbolic link* agar berkas di *folder storage* dapat dibuka dari *browser*:

```
php artisan migrate
php artisan storage:link
```

- storage:link → membuat *folder* pintasan public/storage yang mengarah ke storage/app/public, sehingga berkas dapat dibuka melalui asset('storage/' . $path). Perintah ini cukup dijalankan satu kali untuk setiap *project*.

Gambar 9.29 Hasil php artisan storage:link

5. Buat *factory* untuk *model* Daftar (digunakan pada pengujian otomatis):

```
php artisan module:make-factory Daftar Pendaftaran
```

Buka file Modules/Pendaftaran/database/factories/DaftarFactory.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\Pendaftaran\Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use Modules\Pendaftaran\Models\Daftar;

/**
 * @extends Factory<Daftar>
 */
class DaftarFactory extends Factory
{
 /**
 * The name of the factory's corresponding model.
 */
 protected $model = Daftar::class;

 /**
 * Define the model's default state.
 *
 * @return array<string, mixed>
 */
 public function definition(): array
 {
 return [
 'nama' => fake()->name(),
 'asal_sekolah' => 'SMA '.fake()->city(),
 'prodi_tujuan' => fake()->randomElement(['Informatika', 'Sistem
Informasi', 'Teknik Elektro']),
 'email' => fake()->unique()->safeEmail(),
 'no_hp' => fake()->numerify('08##########'),
 ];
 }
}
```

6. Buka file Modules/Pendaftaran/app/Models/Daftar.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\Pendaftaran\Models;
```

```
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
 /** @use HasFactory<DaftarFactory> */
 use HasFactory;

 /**
 * The attributes that are mass assignable.
 *
 * @var list<string>
 */
 protected $fillable = [
 'nama',
 'asal_sekolah',
 'prodi_tujuan',
 'email',
 'no_hp',
 'berkas',
 'nama_berkas',
 ];

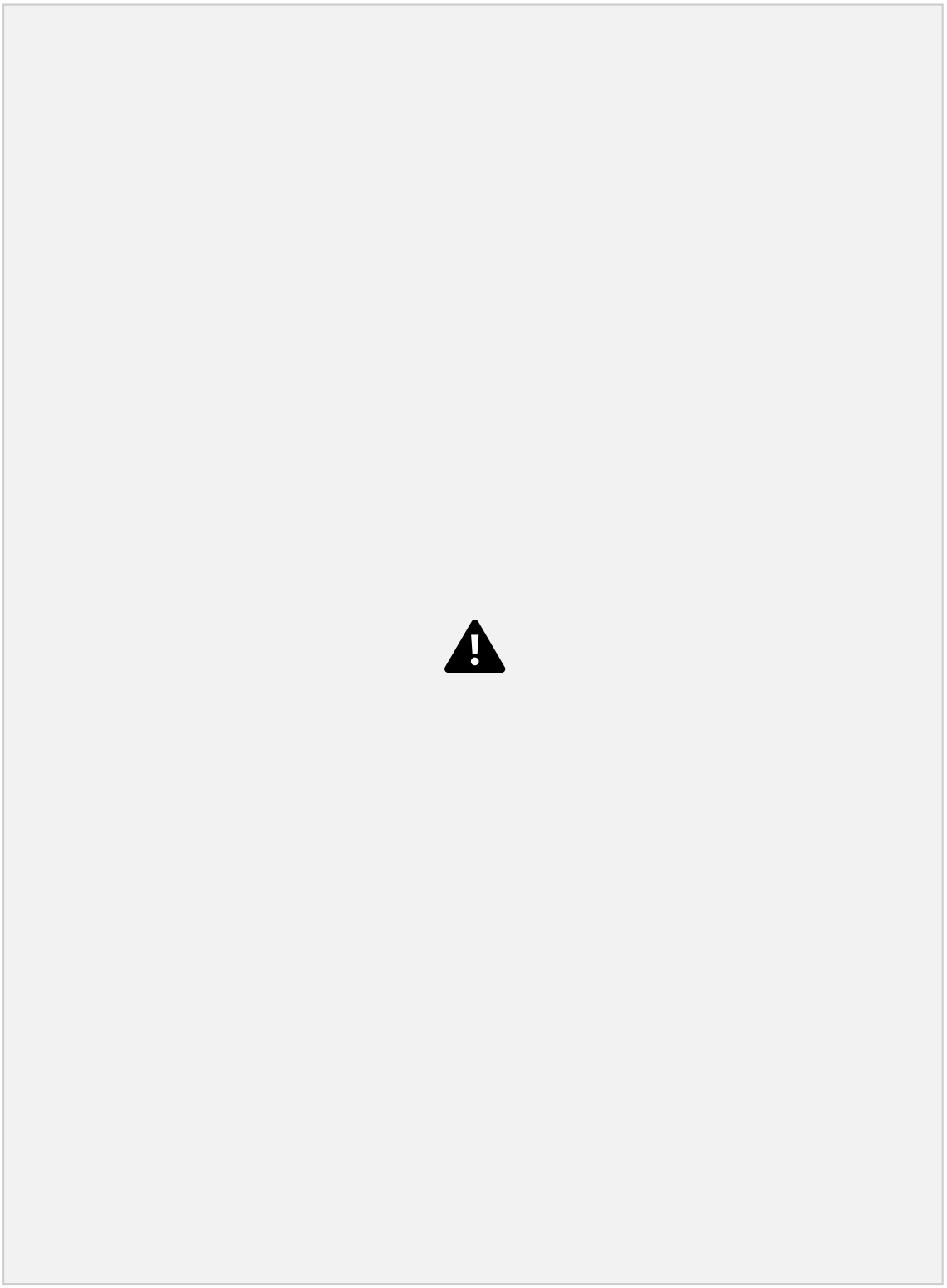
 /**
 * The accessors to append to the model's array and JSON form.
*
 * @var list<string>
 */
 protected $appends = [
 'berkas_url',
 ];

 /**
 * Public URL of the uploaded file, or null when there is no file.
*/
 protected function berkasUrl(): Attribute
 {
 return Attribute::get(fn (): ?string => $this->berkas ?
asset('storage/'.$this->berkas) : null);
 }

 protected static function newFactory(): DaftarFactory
 {
 return DaftarFactory::new();
 }
}
```

• $fillable → lima kolom data ditambah berkas dan nama_berkas yang boleh diisi melalui Daftar::create() dan update().

• newFactory() → menghubungkan *model* modul dengan *factory*-nya, karena *factory* modul berada di *namespace* yang berbeda dari *factory* aplikasi utama. • $appends dan berkasUrl() → menambahkan atribut berkas_url ke data JSON. Nilainya dibuat dengan asset('storage/' . $this->berkas) sehingga React dapat langsung membuat tautan ke berkas tanpa menyusun URL sendiri.



Gambar 9.30 Model Daftar

7. Buat Form Request untuk validasi data pendaftaran:

```
php artisan module:make-request DaftarRequest Pendaftaran
```

Buka file Modules/Pendaftaran/app/Http/Requests/DaftarRequest.php, lalu ganti isinya dengan kode berikut:

```
<?php
```

```
namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
 /**
 * Get the validation rules that apply to the request.
 *
 * The file is required when creating, and optional when updating (keep the old
file).
 *
 * @return array<string, array<int, mixed>>
 */
 public function rules(): array
 {
 $daftar = $this->route('daftar');

 return [
 'nama' => ['required', 'string', 'max:255'],
 'asal_sekolah' => ['required', 'string', 'max:255'],
 'prodi_tujuan' => ['required', 'string', 'max:255'],
 'email' => ['required', 'email', 'max:255', Rule::unique('daftars',
'email')->ignore($daftar?->id)],
 'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
'berkas' => [$daftar ? 'nullable' : 'required', 'file',
'mimes:jpg,jpeg,png,pdf', 'max:2048'],
 ];
 }

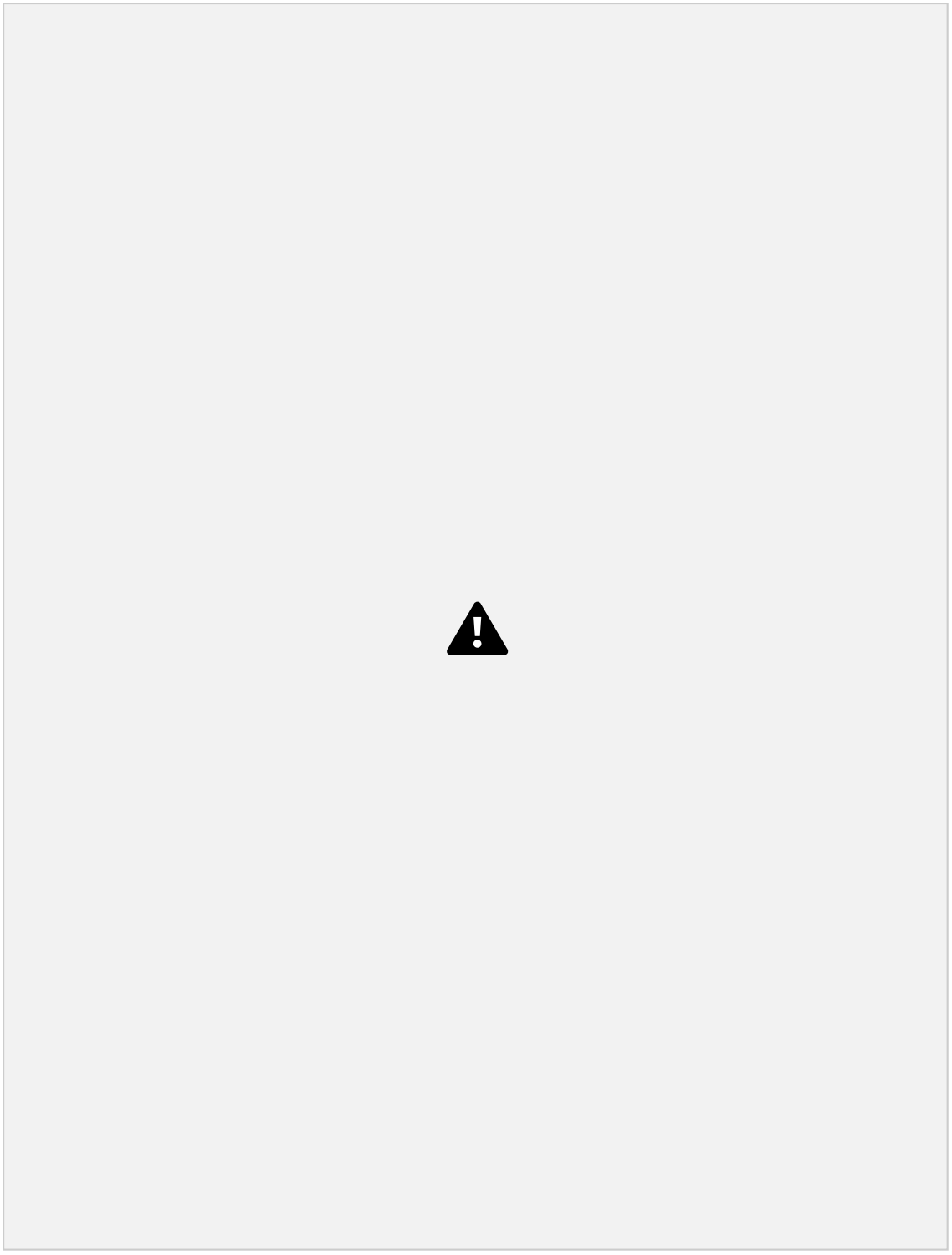
 /**
 * Get custom messages for validator errors.
 *
 * @return array<string, string>
 */
 public function messages(): array
 {
 return [
 'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
'berkas.required' => 'Berkas wajib diunggah.',
 'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
 ];
 }

 /**
 * Determine if the user is authorized to make this request.
 */
 public function authorize(): bool
 {
 return true;
 }
}
```

• Satu *request* dipakai untuk tambah dan edit. ignore($this->route('daftar')?->id) membuat email milik data yang sedang diedit tidak dianggap duplikat. • regex:/^[0-9+]{8,20}$/ → nomor HP hanya boleh berisi angka (dan tanda +) sepanjang 8- 20 karakter. Pesan *error*-nya diganti melalui method messages().

• Karena *helper* http.ts mengirim *header* Accept: application/json, *error* validasi dikembalikan sebagai JSON dengan kode 422, bukan *redirect*.

- berkas → wajib diunggah saat menambah data (required) dan boleh dikosongkan saat edit (nullable) karena berkas lama tetap digunakan. Aturan file, mimes:jpg,jpeg,png,pdf, dan max:2048 memastikan yang diunggah benar-benar file dengan format yang diizinkan dan berukuran maksimal 2 MB.
- messages() → mengganti pesan *error* menjadi Bahasa Indonesia, misalnya Berkas harus berformat jpg, jpeg, png, atau pdf.



Gambar 9.31 DaftarRequest

8. Buka file Modules/Pendaftaran/app/Http/Controllers/PendaftaranController.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
 /**
 * Display the React page. Data is loaded via AJAX.
 */
 public function index(): Response
 {
 return Inertia::render('Pendaftaran::index');
 }

 /**
 * Return all registrations as JSON.
 */
 public function getData(): JsonResponse
 {
 return response()->json(Daftar::latest()->get());
 }

 /**
 * Store a newly created registration together with its uploaded file.  */
 public function store(DaftarRequest $request): JsonResponse
 {
 $data = $request->safe()->except('berkas');

 $file = $request->file('berkas');
 $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
$data['nama_berkas'] = $file->getClientOriginalName();

 Daftar::create($data);

 return response()->json(['success' => 'Data berhasil disimpan.']);  }

 /**
 * Return a single registration for the edit form.
 */
 public function edit(Daftar $daftar): JsonResponse
 {
 return response()->json($daftar);
 }

 /**
 * Update the specified registration. The old file is replaced only when a new  one
is uploaded.
 */
 public function update(DaftarRequest $request, Daftar $daftar): JsonResponse  {
 $data = $request->safe()->except('berkas');
```

```
if ($request->hasFile('berkas')) {
 if ($daftar->berkas) {
 Storage::disk('public')->delete($daftar->berkas);
 }

 $file = $request->file('berkas');
 $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
$data['nama_berkas'] = $file->getClientOriginalName();  }

 $daftar->update($data);

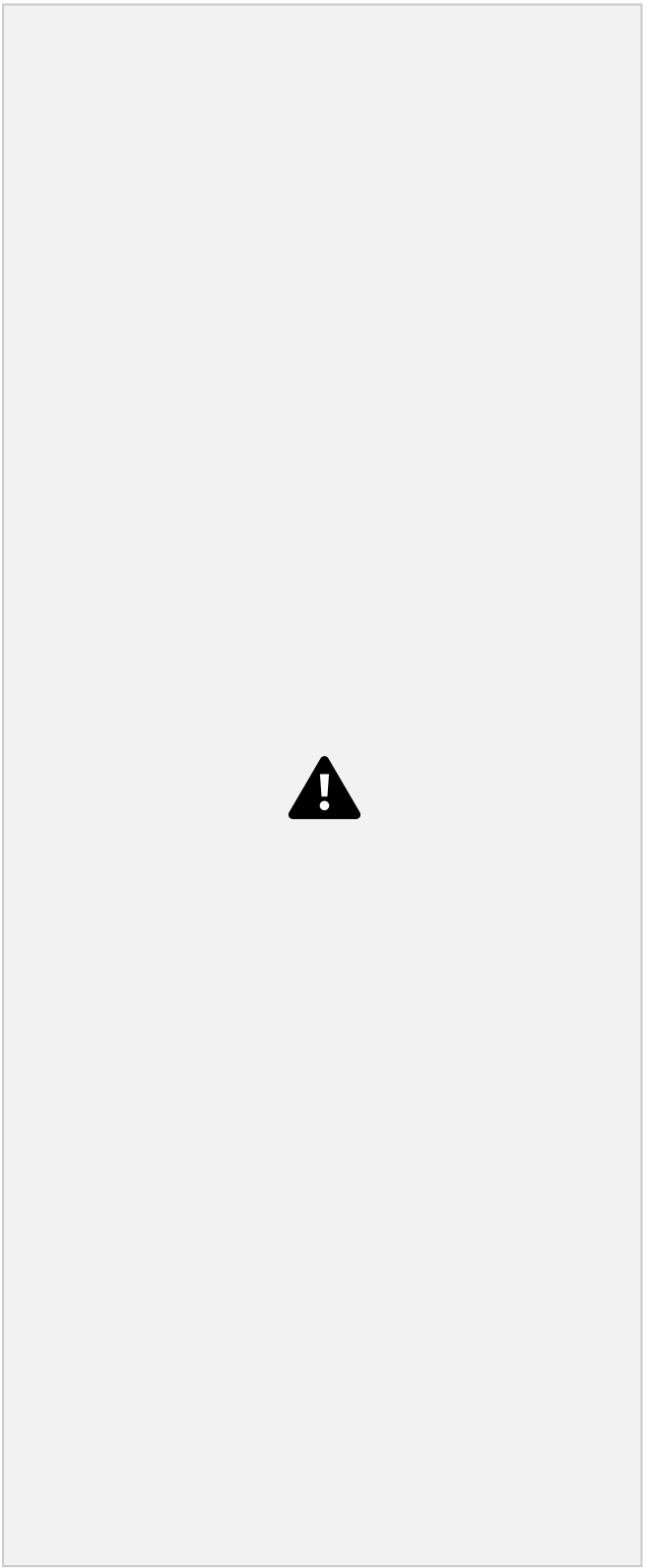
 return response()->json(['success' => 'Data berhasil diperbarui.']);  }

 /**
 * Remove the specified registration and its uploaded file.
 */
 public function destroy(Daftar $daftar): JsonResponse
 {
 if ($daftar->berkas) {
 Storage::disk('public')->delete($daftar->berkas);
 }

 $daftar->delete();

 return response()->json(['success' => 'Data berhasil dihapus.']);  }
}
```

- index() → satu-satunya method yang mengembalikan halaman, yaitu Inertia::render('Pendaftaran::index') tanpa *props* data. Halaman dimuat sekali, lalu data diambil melalui AJAX.
- getData() → mengembalikan seluruh data pendaftar dalam bentuk JSON, diurutkan dari yang terbaru.
- store(), update(), dan destroy() → mengembalikan JSON berisi pesan success yang ditampilkan oleh React.
- edit() → mengembalikan satu data dalam JSON untuk mengisi form di dalam dialog.
- $request->safe()->except('berkas') → mengambil data hasil validasi tanpa objek file, karena yang disimpan ke *database* hanya *path*-nya.
- $file->store('berkas_pendaftaran', 'public') → menyimpan berkas ke storage/app/public/berkas_pendaftaran dengan nama acak yang unik lalu mengembalikan *path*-nya, sedangkan getClientOriginalName() mengambil nama asli file.
- Storage::disk('public')->delete(...) → menghapus berkas lama ketika berkas diganti pada update() dan ketika data dihapus pada destroy(), agar tidak ada file sampah yang tertinggal.
- Daftar $daftar → *route model binding*; jika id tidak ditemukan, Laravel otomatis mengembalikan *error* 404.



Gambar 9.32 PendaftaranController dengan respons JSON dan upload berkas 9.

Buka file Modules/Pendaftaran/routes/web.php, lalu ganti isinya dengan kode berikut:

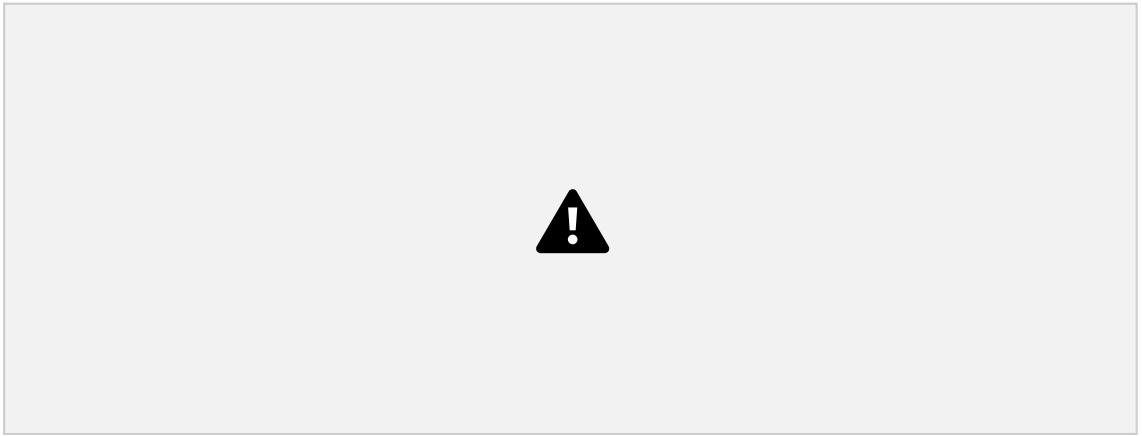
```
<?php

use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;
```



```
Route::middleware(['auth', 'role:admin,user'])->group(function () {
Route::get('pendaftaran', [PendaftaranController::class, 'index'])-
>name('pendaftaran.index');
 Route::get('pendaftaran/data', [PendaftaranController::class, 'getData'])-
>name('pendaftaran.data');
 Route::post('pendaftaran', [PendaftaranController::class, 'store'])-
>name('pendaftaran.store');
 Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class, 'edit'])-
>name('pendaftaran.edit');
 Route::put('pendaftaran/{daftar}', [PendaftaranController::class, 'update'])-
>name('pendaftaran.update');
 Route::delete('pendaftaran/{daftar}', [PendaftaranController::class, 'destroy'])-
>name('pendaftaran.destroy');
});
```

- middleware(['auth', 'role:admin,user']) → modul dapat diakses oleh admin maupun user yang sudah *login*.
- *Route* pendaftaran/data ditulis sebelum *route* pendaftaran/{daftar} agar kata data tidak dianggap sebagai id.
- Edit dan update dipisah: GET pendaftaran/{daftar}/edit mengambil data, sedangkan PUT pendaftaran/{daftar} menyimpan perubahan.



Gambar 9.33 Route modul Pendaftaran

10. Buat *helper* untuk mengirim *request* AJAX. File ini diletakkan di luar modul karena dapat dipakai oleh modul lain. Buat file resources/js/lib/http.ts:

```
export type ValidationErrors = Record<string, string[]>;

type ErrorBody = { message?: string; errors?: ValidationErrors } | null;

export class HttpError extends Error {
 status: number;
 data: ErrorBody;

 constructor(status: number, data: ErrorBody) {
 super(data?.message ?? `Request gagal dengan status ${status}`);
this.status = status;
 this.data = data;
 }
}

// Membaca token CSRF dari cookie XSRF-TOKEN yang dibuat oleh
Laravel function getXsrfToken(): string {
 const cookie = document.cookie.split('; ').find((row) => row.startsWith('XSRF
TOKEN='));
```



```
return cookie ? decodeURIComponent(cookie.substring('XSRF-TOKEN='.length)) : '';
}

type RequestOptions = {
 method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
 body?: FormData;
};

export async function http<T>(url: string, { method = 'GET', body }: RequestOptions
=  {}): Promise<T> {
 const response = await fetch(url, {
 method,
 body,
 credentials: 'same-origin',
 headers: {
 Accept: 'application/json',
 'X-Requested-With': 'XMLHttpRequest',
 'X-XSRF-TOKEN': getXsrfToken(),
 },
 });

 const data = await response.json().catch(() => null);

 if (!response.ok) {
 throw new HttpError(response.status, data);
 }

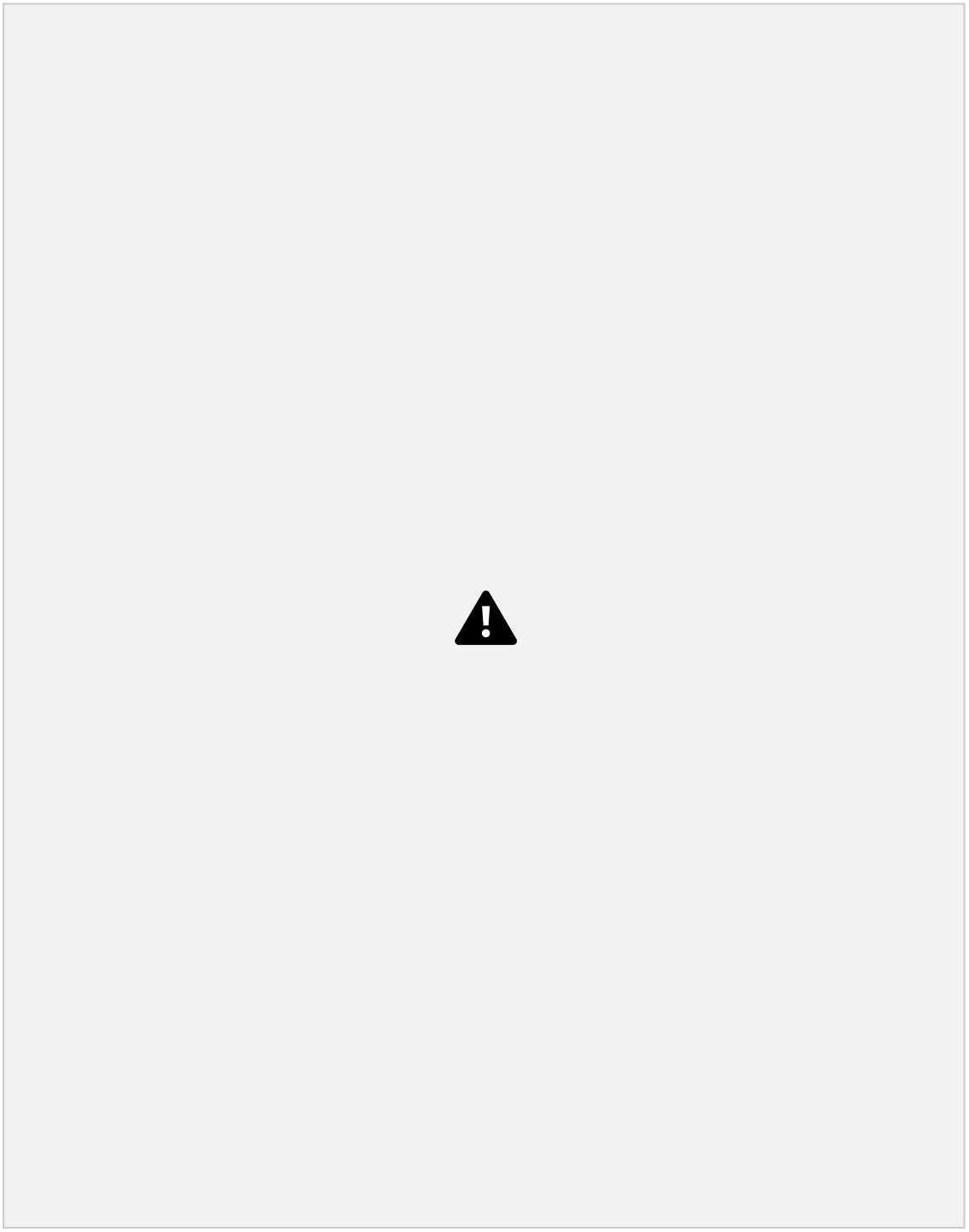
 return data as T;
}
```

- getXsrfToken() → membaca *cookie* XSRF-TOKEN yang otomatis dibuat Laravel pada setiap respons. Nilainya dikirim sebagai *header* X-XSRF-TOKEN (pengganti *header* X CSRF-TOKEN dari tag meta pada $.ajaxSetup() di Modul 8). Tanpa *header* ini *request* POST, PUT, dan DELETE ditolak dengan kode 419.

• Accept: application/json → membuat Laravel mengembalikan *error* validasi sebagai JSON berkode 422 dan *error* lain (403, 404) sebagai JSON, bukan halaman HTML. • headers tidak berisi Content-Type → jika body berisi FormData, *browser* otomatis

mengisi Content-Type: multipart/form-data beserta *boundary*. Ini setara dengan processData: false dan contentType: false pada jQuery.

- if (!response.ok) throw new HttpError(...) → fetch() tidak menganggap kode 4xx/5xx sebagai kegagalan, sehingga status diperiksa manual. HttpError membawa status dan isi JSON agar halaman dapat membedakan *error* 422 dari *error* lainnya.



Gambar 9.34 Helper http.ts untuk request AJAX

11. Buat file Modules/Pendaftaran/resources/js/pages/index.tsx, lalu isi dengan kode berikut:

```
import { Head } from '@inertiajs/react';
import { FileImage, FileText, LoaderCircle, Pencil, Plus, Search, Trash2 }
from  'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import type { ChangeEvent, FormEvent } from 'react';
import { AlertMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import InputError from '@/components/input-error';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
```

```
import {
 Dialog,
 DialogContent,
 DialogDescription,
 DialogFooter,
 DialogHeader,
 DialogTitle,
} from '@/components/ui/dialog';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import AppLayout from '@/layouts/app-layout';
import { http, HttpError } from '@/lib/http';
import type { ValidationErrors } from '@/lib/http';
import type { BreadcrumbItem } from '@/types';

type Daftar = {
 id: number;
 nama: string;
 asal_sekolah: string;
 prodi_tujuan: string;
 email: string;
 no_hp: string;
 berkas: string | null;
 nama_berkas: string | null;
 berkas_url: string | null;
};

type FormFields = Pick<Daftar, 'nama' | 'asal_sekolah' | 'prodi_tujuan' | 'email' |
 'no_hp'>;

type Message = { type: 'success' | 'error'; text: string } | null;

const emptyForm: FormFields = { nama: '', asal_sekolah: '', prodi_tujuan: '', email:
 '', no_hp: '' };

const fields: { name: keyof FormFields; label: string; type?: string }[] =
[  { name: 'nama', label: 'Nama' },
 { name: 'asal_sekolah', label: 'Asal Sekolah' },
 { name: 'prodi_tujuan', label: 'Prodi Tujuan' },
 { name: 'email', label: 'Email', type: 'email' },
 { name: 'no_hp', label: 'No HP' },
];

const urls = {
 data: '/pendaftaran/data',
 store: '/pendaftaran',
 edit: (id: number) => `/pendaftaran/${id}/edit`,
 update: (id: number) => `/pendaftaran/${id}`,
 destroy: (id: number) => `/pendaftaran/${id}`,
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Pendaftaran', href: '/pendaftaran'
}];

// Tautan berkas yang dibuka di tab baru, ikon dipilih berdasarkan ekstensi
function FileLink({ url, name }: { url: string | null; name: string | null }) {
if (!url) {
 return <span className="text-muted-foreground">-</span>;  }

 const Icon = name?.toLowerCase().endsWith('.pdf') ? FileText : FileImage;

 return (
 <a
 href={url}
```

```
target="_blank"
 rel="noopener noreferrer"
 className="inline-flex items-center gap-1 text-primary underline-offset-4
hover:underline"
 >
 <Icon className="size-4" /> {name ?? 'Lihat berkas'}
 </a>
 );
}

export default function Index() {
 const [records, setRecords] = useState<Daftar[]>([]);
 const [loading, setLoading] = useState(true);
 const [keyword, setKeyword] = useState('');
 const [message, setMessage] = useState<Message>(null);

 const [dialogOpen, setDialogOpen] = useState(false);
 const [editing, setEditing] = useState<Daftar | null>(null);
 const [form, setForm] = useState<FormFields>(emptyForm);
 const [file, setFile] = useState<File | null>(null);
 const [fileInputKey, setFileInputKey] = useState(0);
 const [errors, setErrors] = useState<ValidationErrors>({});
 const [saving, setSaving] = useState(false);

 const [deleting, setDeleting] = useState<Daftar | null>(null);
const [processingDelete, setProcessingDelete] = useState(false);

 const showError = (error: unknown, fallback: string) => {
 const text = error instanceof HttpError ? (error.data?.message ?? fallback) :
fallback;
 setMessage({ type: 'error', text });
 };

 // Mengambil seluruh data pendaftar dari server (GET /pendaftaran/data)
const loadData = async () => {
 setLoading(true);

 try {
 setRecords(await http<Daftar[]>(urls.data));
 } catch (error) {
 showError(error, 'Gagal memuat data pendaftar.');
 } finally {
 setLoading(false);
 }
 };

 useEffect(() => {
 loadData();
 }, []);

 // Pencarian dilakukan di browser tanpa request baru ke server
const filtered = useMemo(() => {
 const q = keyword.trim().toLowerCase();

 if (q === '') {
 return records;
 }

 return records.filter((item) =>
 [item.nama, item.asal_sekolah, item.prodi_tujuan, item.email,
item.no_hp].some((value) =>
 value.toLowerCase().includes(q),
 ),
 );
```

```
 }, [records, keyword]);
```



```
const resetFileAndErrors = () => {
 setFile(null);
 setFileInputKey((key) => key + 1);
 setErrors({});
 };

 const openCreate = () => {
 setEditing(null);
 setForm(emptyForm);
 resetFileAndErrors();
 setDialogOpen(true);
 };

 // Mengambil satu data (GET /pendaftaran/{id}/edit) lalu mengisi form di dialog
const openEdit = async (id: number) => {
 try {
 const data = await http<Daftar>(urls.edit(id));

 setEditing(data);
 setForm({
 nama: data.nama,
 asal_sekolah: data.asal_sekolah,
 prodi_tujuan: data.prodi_tujuan,
 email: data.email,
 no_hp: data.no_hp,
 });
 resetFileAndErrors();
 setDialogOpen(true);
 } catch (error) {
 showError(error, 'Data tidak ditemukan.');
 }
 };

 const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
setForm({ ...form, [event.target.name]: event.target.value });  };

 // Menyimpan data (tambah dan edit) menggunakan FormData
 const handleSubmit = async (event: FormEvent) => {
 event.preventDefault();
 setSaving(true);
 setErrors({});

 const formData = new FormData();
        Object.entries(form).forEach(([key, value]) => formData.append(key, value));

 if (file) {
 formData.append('berkas', file);
 }

 if (editing) {
 formData.append('_method', 'PUT');
 }

 try {
 const result = await http<{ success: string }>(editing ?
urls.update(editing.id) : urls.store, {
 method: 'POST',
 body: formData,
 });

 setDialogOpen(false);
 setMessage({ type: 'success', text: result.success });  await
loadData();
```

```
} catch (error) {
 if (error instanceof HttpError && error.status === 422) {
setErrors(error.data?.errors ?? {});
 } else {
 showError(error, 'Terjadi kesalahan, silakan coba lagi.');  }
 } finally {
 setSaving(false);
 }
 };

 // Menghapus data (DELETE /pendaftaran/{id}) setelah dikonfirmasi
const handleDelete = async () => {
 if (!deleting) {
 return;
 }

 setProcessingDelete(true);

 try {
 const result = await http<{ success: string }>(urls.destroy(deleting.id),  { method:
'DELETE' });

 setMessage({ type: 'success', text: result.success });  await
loadData();
 } catch (error) {
 showError(error, 'Data gagal dihapus.');
 } finally {
 setProcessingDelete(false);
 setDeleting(null);
 }
 };

 return (
 <AppLayout breadcrumbs={breadcrumbs}>
 <Head title="Pendaftaran" />

 <div className="flex flex-1 flex-col gap-4 p-4">
 <div className="flex flex-wrap items-center justify-between gap-2">  <div>
 <h1 className="flex items-center gap-2 text-xl font semibold">
 Data Pendaftar <Badge
variant="secondary">{records.length}</Badge>
 </h1>
                        <p className="text-sm text-muted-foreground">
 Seluruh operasi pada halaman ini berjalan dengan AJAX  tanpa memuat ulang
halaman.
 </p>
 </div>
                    <Button onClick={openCreate}>
 <Plus /> Tambah Pendaftar
 </Button>
 </div>

 <AlertMessage type={message?.type ?? 'success'}
message={message?.text} onClose={() => setMessage(null)} />

 <div className="relative max-w-sm">
 <Search className="absolute top-1/2 left-3 size-4 -translate-y 1/2
text-muted-foreground" />
 <Input
 value={keyword}
                        onChange={(event) => setKeyword(event.target.value)}
                        placeholder="Cari nama, sekolah, prodi, email..."
```

```
className="pl-9"
 />
 </div>

 <div className="overflow-x-auto rounded-xl border">  <table
className="w-full text-sm">
 <thead className="bg-muted/50 text-left">  <tr>
 <th className="px-4 py-3 font-medium">No</th>  <th className="px-4 py-3
font-medium">Nama</th>  <th className="px-4 py-3 font-medium">Asal
Sekolah</th>
 <th className="px-4 py-3 font-medium">Prodi  Tujuan</th>
 <th className="px-4 py-3 font-medium">Email</th>  <th className="px-4 py-3
font-medium">No HP</th>  <th className="px-4 py-3 font-medium">Berkas</th>  <th
className="px-4 py-3 text-right font medium">Aksi</th>
 </tr>
 </thead>
                        <tbody>
 {loading && (
 <tr>
 <td colSpan={8} className="px-4 py-8 text-center  text-muted-foreground">
 <LoaderCircle className="mx-auto size-5  animate-spin" />
 </td>
 </tr>
 )}
                            {!loading && filtered.length === 0 && (
 <tr>
 <td colSpan={8} className="px-4 py-8 text-center  text-muted-foreground">
 {records.length === 0 ? 'Belum ada data  pendaftar.' : 'Data tidak ditemukan.'}
 </td>
 </tr>
 )}
                            {!loading &&
 filtered.map((item, index) => (
 <tr key={item.id} className="border-t">  <td className="px-4 py-3">{index +
1}</td>  <td className="px-4 py-3 font medium">{item.nama}</td>
 <td className="px-4 py
3">{item.asal_sekolah}</td>
 <td className="px-4 py
3">{item.prodi_tujuan}</td>
 <td className="px-4 py-3">{item.email}</td>  <td className="px-4
py-3">{item.no_hp}</td>  <td className="px-4 py-3">  <FileLink
url={item.berkas_url}  name={item.nama_berkas} />
 </td>
                                        <td className="px-4 py-3">
  <div  className="flex  justify-end  gap-2">    <Button  size="sm"  variant="outline"
onClick={() => openEdit(item.id)}>
 <Pencil /> Edit  </Button>
                                                  <Button size="sm"
variant="destructive" onClick={() => setDeleting(item)}>
 <Trash2 /> Hapus
```



```
</Button>
 </div>
 </td>
 </tr>
 )}
 </tbody>
 </table>
 </div>
 </div>
```

```
 <Dialog open={dialogOpen} onOpenChange={(open) => !saving &&
setDialogOpen(open)}>
 <DialogContent className="sm:max-w-lg">
 <form onSubmit={handleSubmit} noValidate className="space-y-4">  <DialogHeader>
 <DialogTitle>{editing ? 'Edit Pendaftar' : 'Tambah  Pendaftar'}</DialogTitle>
 <DialogDescription>Lengkapi data calon mahasiswa beserta  berkas
pendukungnya.</DialogDescription>
 </DialogHeader>

 {fields.map((field) => (
 <div key={field.name} className="grid gap-2">  <Label
htmlFor={field.name}>
 {field.label} <span className="text-red 500">*</span>
 </Label>
                                  <Input
 id={field.name}
                                      name={field.name}
                                      type={field.type ?? 'text'}
                                      value={form[field.name]}
                                      onChange={handleChange}
                                      aria-invalid={!!errors[field.name]}
 />
                                  <InputError message={errors[field.name]?.[0]} />
 </div>
 ))}

 <div className="grid gap-2">
 <Label htmlFor="berkas">
 Berkas Pendukung {!editing && <span className="text red-500">*</span>}
 </Label>
                              <Input
 key={fileInputKey}
                                  id="berkas"
                                  type="file"
                                  accept=".jpg,.jpeg,.png,.pdf"
                                   onChange={(event) => setFile(event.target.files?.[0]
?? null)}
 aria-invalid={!!errors.berkas}
 />
                              <p className="text-xs text-muted-foreground">
 Format jpg, jpeg, png, atau pdf, maksimal 2 MB. {editing && ' Kosongkan jika tidak
                                  ingin mengganti
berkas.'}
 </p>
                             {editing?.berkas_url && (
 <p className="text-xs">
 Berkas saat ini: <FileLink
url={editing.berkas_url} name={editing.nama_berkas} />
 </p>
 )}
                              <InputError message={errors.berkas?.[0]} />
```

```
</div>

 <DialogFooter className="gap-2">
 <Button type="button" variant="secondary" onClick={() =>  setDialogOpen(false)}
disabled={saving}>
 Batal
 </Button>
                          <Button type="submit" disabled={saving}>
  {saving && <LoaderCircle className="animate-spin" />} {saving ? 'Menyimpan...' :
                                    'Simpan'}
 </Button>
 </DialogFooter>
 </form>
 </DialogContent>
 </Dialog>

 <ConfirmDialog
 open={deleting !== null}
 description={`Yakin ingin menghapus data ${deleting?.nama} beserta
berkasnya?`}
 processing={processingDelete}
 onConfirm={handleDelete}
 onOpenChange={(open) => !open && setDeleting(null)}  />
 </AppLayout>
 );
}
```

Kode di atas cukup panjang, sehingga penjelasannya dibagi menjadi beberapa bagian berikut.